

A first-order meta learning method for remaining useful life prediction of rotating machinery under limited samples

Yu Wang^{a,b}, Shujie Liu^{a,b,c,*}, Shuai Lv^{a,b}, Gengshuo Liu^{a,b}

^a School of Mechanical Engineering, Dalian University of Technology, Dalian 116024, China

^b State Key Laboratory of High-performance Precision Manufacturing, Dalian University of Technology, Dalian 116024, China

^c Ningbo Institute of Dalian University of Technology, Ningbo 315000, China

HIGHLIGHTS

- A conformer-based learner combining self-attention mechanism and convolution was established.
- A dual-loop meta-learning strategy was constructed to learn at both the task and sample levels.
- An adaptive weighted loss was constructed to deal with the imbalance of labels in extreme sample scarcity scenarios.
- The effectiveness of the proposed method was verified using real industrial data.

ARTICLE INFO

Keywords:

Remaining useful life
Conformer
Meta-learning
Few-shot learning

ABSTRACT

Accurately predicting the remaining useful life (RUL) of rotating machinery is a challenging task in the field of Prognostics and Health Management (PHM). In practical applications, the number of samples in the target domain is often insufficient. To address this issue, we propose a First-Order Meta-Learning Network (FOMLN) to tackle the problem of equipment RUL prediction under limited samples. First, a meta-learner is constructed based on Conformer, combining the advantages of the self-attention mechanism and convolutional neural networks, enhancing the model's ability to capture both local and global features. Then, a dual-loop meta-learning strategy is designed: the inner loop learns at the sample level, modeling and updating parameters for specific tasks, while the outer loop updates the meta-parameters through task-level learning, improving the model's generalization across different tasks and its adaptability to new tasks under limited sample conditions. Extensive experimental results on the C-MAPSS dataset validate the effectiveness of the proposed method. Moreover, a practical application case study is introduced, demonstrating the model's ability to predict the RUL of slurry pumps in an industrial site under few-shot scenarios, highlighting its potential for real-world applications.

1. Introduction

In the field of prognostic health management (PHM), accurately estimating the remaining useful life (RUL) of machinery is a critical task [1], serving as the foundation for predictive maintenance and operational decision-making. The primary objective of RUL prediction is to forecast the potential failure time of equipment, thus enabling preemptive maintenance and enhancing reliability. In recent years, this topic has garnered considerable attention from researchers, leading to substantial advancements [2].

With the progress in information technology, machine learning (ML) techniques have gained prominence in RUL prediction [3]. Notably,

methods based on convolutional neural networks (CNN), Transformer models [4], and recurrent neural networks (RNN) such as long short-term memory (LSTM) networks [5] and gated recurrent units (GRU) [6] have shown promise. These ML approaches excel at automatically extracting features from complex data and handling high-dimensional, nonlinear datasets [7]. For instance, Hu et al. [8] integrated CNN with quantile regression to quantify the uncertainty in RUL predictions. Similarly, Liang et al. [9] introduced a Bayesian framework into GRU to improve the accuracy of RUL estimation.

While ML-based RUL prediction methods have made considerable progress, they still face serious challenges in real-world industrial deployment [10]. Firstly, these models struggle to effectively capture

* Corresponding author at: School of Mechanical Engineering, Dalian University of Technology, Dalian 116024, China.

E-mail address: liushujie@dlut.edu.cn (S. Liu).

<https://doi.org/10.1016/j.asoc.2025.113616>

Received 29 October 2024; Received in revised form 11 June 2025; Accepted 9 July 2025

Available online 15 July 2025

1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

the rich multi-scale degradation features present in sensor signals. Existing approaches often exhibit trade-offs: RNN-like models insufficiently model long-range dependencies, CNNs have difficulty incorporating global context, and Transformers are less efficient in extracting local features. Therefore, designing a method that can extract and fuse both local and global features from raw sensor data to construct a comprehensive feature representation highly predictive of RUL is a crucial bottleneck for improving the performance of current RUL prediction models.

The second challenge lies in the domain shift problem [11,12], particularly when a model is trained in one environment and deployed in a different one. Domain shift can be caused by a variety of factors, such as different operating conditions, equipment wear status, or environmental changes [13]. This phenomenon poses a challenge to the accuracy of RUL prediction, as the model may not be able to effectively generalize from the training data (source domain) to the actual application environment (target domain) [14]. To address the domain shift problem, domain adaptation (DA) technology has been proposed. DA methods usually require a large number of unlabeled target domain samples to adjust the model so that it can better generalize to new environments [15,16]. For example, Zou et al. [17] proposed a dynamic benchmark (DB)-based transfer prediction model to predict the RUL of bearings, using autoencoders and DB-based domain adaptation methods to reconstruct features. However, the scenarios in real applications are often more demanding, because obtaining a large number of target domain samples is time-consuming and expensive, especially in industrial applications of high-value equipment [18]. Therefore, how to achieve effective cross-domain learning in situations where the number of target domain samples is severely limited has become a critical problem in the RUL prediction field. Although meta-learning is a potential solution to this problem [19], most meta-learning models in the RUL prediction domain are implemented based on the Model-Agnostic Meta-Learning (MAML) [20] framework. While MAML has strong adaptability and theoretical foundations, its training process involves the calculation of second-order gradients of the loss function, which leads to high computational complexity and resource consumption, making it difficult to apply effectively in resource-constrained environments.

To address the limitations outlined above – namely, the insufficient multi-scale feature representation in existing RUL prediction models, the scarcity of target-domain samples in domain-shift scenarios, and the computational drawbacks of current MAML-based meta-learning approaches – this paper proposes a First-Order Meta-Learning Network (FOMLN). The method is based on first-order meta-learning framework [21]. The inner loop updates task-level parameters for a specific meta-task by learning at the level of individual samples. The outer loop, in turn, updates meta-level parameters by aligning the model's parameters with aggregated, optimal parameters derived from each meta-task. Moreover, to facilitate effective extraction of multi-scale degradation features, we developed a meta-learner based on a Conformer [22] architecture. This meta-learner effectively merges the local feature extraction strengths of CNNs with the aptitude of Transformers for processing long-range sequence dependencies. By integrating this meta-learner into the FOMLN framework, our model is capable of learning robust and generalizable feature representations, even from limited data, enabling rapid adaptation to novel, data-scarce target domains.

The main contributions of this paper are as follows:

1. An efficient first-order episodic meta-training strategy for FOMLN is proposed. This strategy relies on the first-order derivative of the loss function to perform inner (task-level parameter update) and outer (meta-level parameter update) loop optimization, reducing computational complexity. This enables FOMLN to be effectively trained with limited samples and rapidly adapted to new RUL prediction tasks with only a small number of target samples.
2. A Conformer-based meta-learner has been constructed and integrated into the FOMLN framework. By combining the local perception capability of CNNs and the global modeling capability of Transformers, this learner effectively addresses the challenge of extracting multi-scale features from device degradation signals.
3. Extensive experimental validation on the public benchmark dataset C-MAPSS, particularly in the few-shot cross-domain RUL prediction setting that has received less attention in previous studies, demonstrates the superiority of the proposed method. Furthermore, a case study predicting the RUL of a slurry pump in an industrial field environment further showcases the practical application potential and value of FOMLN in real-world few-shot scenarios.

The rest of the paper is organized as follows: Section 2 includes related work on the method, Section 3 presents the overall structure and training method of the model, and related experiments are conducted in Section 4. Finally, Section 5 summarizes the entire paper.

2. Related work

2.1. Multi-scale learning-based RUL prediction

Multi-scale learning in RUL prediction aims to achieve a more comprehensive feature representation by extracting representative degradation features at different temporal or spatial scales. For instance, Xu et al. [23] extracted multi-scale features from raw sensor signals using a multi-scale temporal convolutional network and combined them with an attention mechanism to predict the RUL of aero-engines. Guo et al. [24] addressed the challenges of capturing multi-scale degradation trends and extracting long-term dependencies in aero-engine RUL prediction by combining a multi-scale feature extractor and a Transformer network, constructing an Hourglass-Transformer model (MHT). They validated the method's superiority and generalizability on the C-MAPSS and N-CMAPSS datasets. Zhou et al. [25] solved the problem of separating RUL prediction and Health State (HS) assessment tasks in aero-engines, as well as the limitations of single-scale feature extraction, by combining multi-scale feature fusion and multi-task joint learning, constructing a multi-task convolutional neural network (MSMT-CNN). Li et al. [26] constructed a Dual-Scale Transformer model (DSFormer) by combining a dual-attention mechanism and a temporal convolutional network (TCN), addressing the impact of sensor differences and the loss of location information in multivariate time-series RUL prediction in the Industrial Internet of Things (IIoT) environment. While the above studies have all validated the effectiveness of their multi-scale feature extraction methods on public benchmark datasets such as C-MAPSS, these works have largely been achieved under relatively data-rich conditions. In real-world industrial scenarios, degradation data, especially near the failure stage of equipment's life cycle, is often very scarce. This

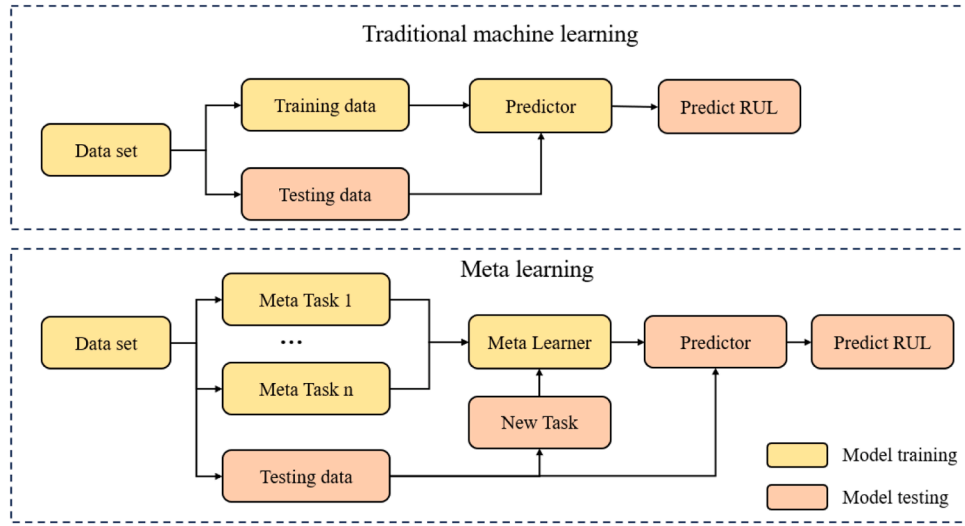


Fig. 1. The difference between meta-learning and traditional machine learning in RUL prediction tasks.

presents a challenge for training these complex models that rely on copious data for multi-scale feature learning, limiting their direct application potential in actual industrial environments.

2.2. Meta-learning and few-shot learning-based RUL prediction

Meta-learning [27], or the concept of "learning how to learn," focuses on enabling models to rapidly adapt to new tasks, even when only a

small number of data points are available for the model to adapt to. Therefore, meta-learning is an important approach to solving few-shot learning (FSL) problems. These methods have shown promise in fields such as medical imaging and natural language processing [28]. As illustrated in Fig. 1, meta-learning differs from traditional ML by enabling models to learn to quickly adapt to new tasks, allowing them to achieve effective generalization with limited target domain data [27]. This is particularly valuable in data-scarce environments; therefore,

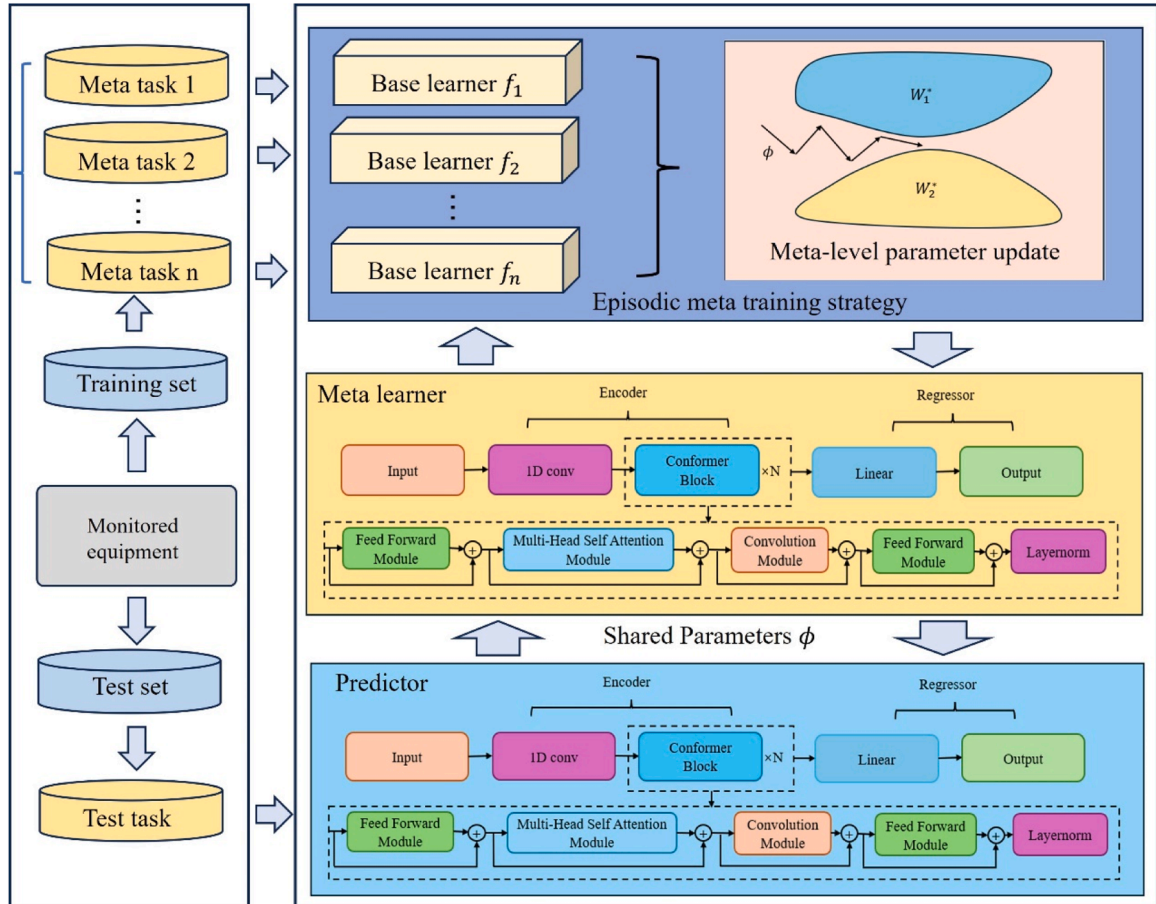


Fig. 2. The overall process of the proposed FOMLN.

meta-learning holds promise as a powerful tool for addressing domain shift issues in few-shot RUL prediction [29–31]. Several studies have applied meta-learning techniques to RUL prediction. For example, Pan et al. [31] combined transfer learning techniques with meta-learning for rocket engine RUL prediction tasks under few-shot scenarios; Yang et al. [32] proposed a Meta-Learning with Deep Flow Kernel Network model for RUL prediction under the few shot and cross-domain conditions, and verified the effectiveness of the proposed method on degradation datasets of aircraft engines and bearings; Zhang et al. [33] proposed an adaptive gradient hybrid feature boundary enhanced meta-learning network for tool wear prediction. Chang et al. [34] proposed a Bayesian model-agnostic meta-learning model with predictive uncertainty calibration for probabilistic RUL prediction.

While the above MAML-based meta-learning methods demonstrate good performance in few-shot cross-domain RUL prediction tasks, these approaches require the calculation of second-order derivatives with respect to model parameters during the meta-training phase, resulting in extremely time-consuming training processes. Furthermore, the second-order derivative calculation itself has poor numerical stability and is prone to gradient explosion or vanishing problems, which can affect the model's convergence speed and final performance.

3. Methodology

The overall process of the proposed FOMLN is shown in Fig. 2. Data is collected from the monitored equipment to form a training set and a test set, where the training set is divided into multiple meta-tasks and the test set is treated as a separate test task. For each meta-task in the training set, FOMLN replicates the structure and parameters of its meta-learner to generate a base learner and performs gradient updates on the meta-task through the structure of the encoder and regressor to model the meta-task at the sample level. Then, the first-order episodic meta-training strategy is designed to align the gradient directions of each base learner, enabling meta-level parameter updates on the meta-learner and facilitating task-level learning. In the testing phase, a predictor that shares initial parameters and structure with the meta-learner is generated based on the test task to perform RUL prediction on the test set. This approach allows FOMLN to learn common knowledge across different tasks, enabling rapid adaptation when confronted with new test tasks.

3.1. Construction of Meta-Learner

ML models used for RUL prediction usually consist of an encoder and a predictor. In the construction of the model encoder, the attention mechanism has become a hot topic in recent years because it can capture long-distance dependencies, which is very important when processing complex time series data. However, the attention mechanism usually requires a lot of computing resources and is not as effective as CNN when processing high-frequency data. On the other hand, although CNN performs well in extracting local features and processing high-frequency data, it is not as powerful as the attention mechanism in capturing long-term dependencies. In order to combine the advantages of both, the Conformer [22] was proposed. By combining the advantages of the attention mechanism and convolution, it can not only effectively capture long-term dependencies in time series data, but also process complex frequency domain features [35]. This combination makes the Conformer both efficient and reliable when processing complex industrial data, but to the best of our knowledge, no scholars have yet explored the performance of this architecture in RUL prediction tasks.

In summary, we designed the meta-learner based on Conformer in Fig. 2, which consists of an encoder and a regressor. The encoder part consists of a one-dimensional convolutional layer and a sequence of N Conformer blocks to perform feature extraction on the input data. These features are then passed to the regressor, whose purpose is to map the high-dimensional feature space into the device's RUL. The linear module

in the regressor consists of a flattening operation followed by a fully connected layer. The flattening operation adjusts the dimension of the encoder output to make it suitable for the fully connected layer, which is used to learn and predict the target RUL.

The encoder of the meta-learner consists of a 1D convolution and N Conformer Blocks, where the 1D convolution transforms the input data into a higher-level feature representation and adjusts its dimension for subsequent feature extraction by the conformer block. In the conformer block, the data after the 1D convolution next enters the Feed Forward Module (FFM) to obtain additional representation capabilities; it then enters the multi-head self-attention module to capture the mechanism of complex interactions between different positions in the sequence. It does this by learning information represented by different subspaces in the sequence in parallel. A self-attention layer allows each element in the input sequence to pay attention to (or weight) all other elements in the sequence to better encode the element. In the model proposed in this article, self-attention is calculated through the following steps [36]:

First, the input sequence X is linearly transformed into three different spaces to obtain Query (Q), Key (K), and Value (V):

$$Q = XW^Q, K = XW^K, V = XW^V \quad (1)$$

Where $W^Q, W^K \in R^{d_{model} \times d_k}$ and $W^V \in R^{d_{model} \times d_v}$ are learnable weight matrices, where d_{model} is the model's feature dimension, d_k is the dimension of keys and queries, and d_v is the dimension of values.

Next, the attention weights are calculated, which can be represented as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

Where d_k is the dimension of the key vectors, and the normalization factor $\sqrt{d_k}$ is used to scale the dot product for more stable gradients. The softmax function ensures that all weights sum up to 1.

In Multi-Head Self-Attention, the above process is replicated and executed in parallel multiple times, with each head learning different aspects of the sequence:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O \quad (3)$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$, each head head_i corresponds to a unique set of weight matrices W_i^Q, W_i^K , and W_i^V . The outputs of all heads are concatenated, and this vector is transformed through a weight matrix W^O to produce the final output representation.

The structure of the FFM is shown in Fig. 3. It consists of a layer normalization, two linear layers, two dropout layers, and a swish activation. Assume that its input is X_2 , which is first processed by layer normalization to stabilize the input distribution of subsequent layers, speed up training, and improve the convergence performance of the model. Unlike Batch Normalization that normalizes along the batch dimension, Layer Normalization normalizes along the feature dimension. This process can be represented as:

$$X_{\text{norm}} = \frac{X_2 - E[X_2]}{\sqrt{\text{Var}[X_2] + \epsilon}} \odot \gamma + \beta \quad (4)$$

Where $E[X_2]$ and $\text{Var}[X_2]$ are the mean and variance of X_2 , respectively, ϵ is a small constant, and γ and β are learnable scale and shift parameters. $\odot$ represents element-wise multiplication.

The normalized data is passed through the linear layer to learn the complex relationship and representation between features. After that,

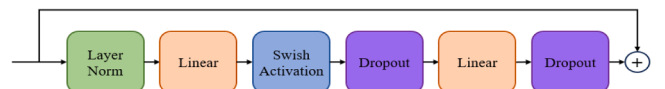


Fig. 3. Structure of Feed Forward Module.

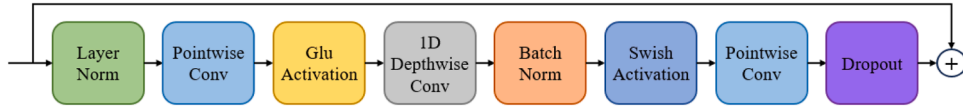


Fig. 4. Structure of Convolution Module.

Swish Activation is used to enhance its nonlinear expression ability. Assuming X_3 is the output of the linear layer, Swish Activation can be expressed as

$$\text{Swish}(X_3) = X_3 \cdot \sigma(X_3) \quad (5)$$

where X_3 is the input and $\sigma(X_3)$ is the sigmoid function.

After passing through the Swish Activation, the data undergoes dropout, linear, and another dropout transformation in succession. Finally, a residual connection adds the original input X_2 of the FFM to the output of Dropout, combining the input and processed features. This process can be represented as:

$$X_{FFM} = FFM(X_2) + X_2 \quad (6)$$

Here, $FFM(X_2)$ represents the output after X_2 has passed through LayerNorm, the first Linear layer, Swish activation, Dropout, the second Linear layer, and the final Dropout within the FFM.

The structure of Convolution Module (CM) is shown in Fig. 4. The data is processed by Layer Norm, Pointwise Conv, Glu Activation, 1D Depthwise Conv, Batch Norm, Swish Activation, Pointwise Conv, and Dropout. Finally, the residual connection adds the original input of CM to the output of Dropout to obtain the final features. Pointwise Conv is a convolution of a single point with a convolution kernel size of 1×1 . The main function of point-by-point convolution is to change the depth of the data without changing the spatial dimension. Then, the nonlinear expression of the model is increased through the GLU (Gated Linear Unit) activation function. Assuming that the input of CM is X_4 , this process can be expressed as

$$\text{GLU}(X_4) = X_4 * w_{\text{pointwise}} \odot \sigma(X_4 * w_{\text{pointwise}}) \quad (7)$$

Where $*$ represents the convolution operation, $w_{\text{pointwise}}$ is the convolution kernel weight, σ is the sigmoid activation function, and $\odot$ represents element-wise multiplication.

After GLU activation, the data $Y_4 = \text{GLU}(X_4)$ is further extracted by 1D Depthwise Convolution. This process applies independent convolution to each input channel instead of mixing information across channels. Each convolution kernel performs convolution operation only on one corresponding channel. 1D Depthwise Convolution can be expressed as:

$$Y_4^c = W_c * Y_4^c + b_c \text{ for each channel } c \quad (8)$$

Where $*$ represents the convolution operation, Y_4^c represents the c th channel of Y_4 , W_c is the convolution kernel of this channel, and b_c is the corresponding bias term.

Then, after the data has been batch normalized, swish activated, Pointwise Convolution, and dropout, the residual connection adds the original input X_4 of CM to the output of Dropout, that is:

$$X_{CM} = \text{CM}(X_4) + X_4 \quad (9)$$

Here, $M(X_4)$ denotes the output after X_4 passes through LayerNorm,

PointwiseConv, GLU, DepthwiseConv, BatchNorm, Swish, the second PointwiseConv, and Dropout.

3.2. Adaptive weighted loss function

In actual operation, equipment is in health state for most of the time, and only shows degradation trends when it is close to failure. Therefore, the number of samples with a relatively long RUL or health state is relatively large. This leads to an imbalanced distribution of labels in the dataset when using equipment degradation data to train neural networks, causing the model's prediction results to be biased towards the majority class in the data [37], that is, samples with longer RUL. This situation will cause the model's performance to deteriorate when predicting shorter RUL, because the model does not "see" such samples enough during the training process [38].

This label imbalance problem is particularly significant in few-shot scenarios. In the meta-learning process, the number of samples in each meta-task is extremely small, which makes the label imbalance problem more prominent than in traditional machine learning models. In this case, the model is more susceptible to label imbalance because the proportion of minority class labels present in a small number of samples is further reduced. This causes the model to learn frequently occurring situations most of the time and ignore all situations, resulting in an inefficient learning process and poor performance when encountering minority class samples.

To address the problem of label imbalance in training data, we propose an adaptive weighted loss function so that different RUL samples in the same meta-batch contribute differently to the loss. This weighted loss function usually gives higher weight to minority samples (samples with shorter RUL) and lower weight to majority samples (samples with longer RUL). This design balances the impact of a few labels on model training by dynamically adjusting weights in the internal and external loops of meta-learning. This loss can be expressed as:

$$L = \frac{1}{N} \sum_{i=1}^N \left(\frac{\max(\text{freq}(y))}{\text{freq}(y_i) + \lambda} \right)^\beta \cdot (\hat{y}_i - y_i)^2 \quad (10)$$

Here, N represents the total number of samples, and β is a tuning exponent that controls the intensity of the weights distribution, set to 1 in this research. λ is a small smoothing factor, chosen to be 1 in this study, primarily to prevent potential division by zero and to provide minor smoothing to the frequency estimate. $\text{freq}(y_i)$ denotes the frequency of samples with an RUL value equal to y_i within the batch, calculated as:

$$\text{freq}(y_i) = \frac{N_{y=y_i}}{N} \quad (11)$$

Where $N_{y=y_i}$ is the count of samples in the current batch whose true RUL is exactly y_i . N is the total number of samples in the batch.

This adaptive weighted loss function assigns a higher weight $w_i =$

$\left(\frac{\max(\text{freq}(y))}{\text{freq}(y_i) + \lambda}\right)^\beta$ to samples i with less frequent RUL values (y_i), their contribution to the overall batch gradient $\nabla_\phi L = \frac{2}{N} \sum w_i (\hat{y}_i - y_i) \nabla_\phi \hat{y}_i$ is amplified. Conversely, the influence of high-frequency samples is attenuated. This re-balancing shifts the gradient direction, compelling the optimizer to pay greater attention to reducing errors on the under-represented but often more critical low-RUL samples. Consequently, the model is encouraged to learn features predictive across the entire degradation process, leading to a more robust convergence towards parameters that generalize better in few-shot scenarios where label imbalance within tasks can be pronounced.

3.3. Meta parameters update framework

3.3.1. First order episodic meta training strategy

The core idea of the proposed FOMLN is to find a good global parameter initialization by simulating the fast adaptation process on multiple different tasks. This initialized parameter should be able to quickly adapt to new tasks within a small number of gradient update steps [39]. By performing this "training of training" on multiple tasks, the model can find an optimal starting point that can generalize to new tasks. Based on this idea, we design a first order episodic meta training strategy for meta-level learning and parameter updating.

Let the meta-learner be f , and the goal of FOMLN is to find a set of parameters ϕ of f so that the model can quickly adapt to a series of tasks $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$. We use the Euclidean distance to measure the closeness of ϕ to the optimal parameter set $\mathcal{W}_T$ for all tasks $T \in \mathcal{T}$ in the parameter space. Therefore, our goal is to find the parameters ϕ so that for all $T \in \mathcal{T}$, the distance $D(\phi, \mathcal{W}_T)$ is small. That is, minimize the following expectation:

$$\min_{\phi} E_T \left[\frac{1}{2} D(\phi, \mathcal{W}_T)^2 \right] \quad (12)$$

Among them, $D(\phi, \mathcal{W}_T)$ represents the Euclidean distance from ϕ to the set $\mathcal{W}_T$.

For a non-pathological set SCR^d , the gradient of the squared distance $D(\phi, S)^2$ with respect to ϕ is $2(\phi - P_S(\phi))$ for almost all points $\phi \in R^d$, where $P_S(\phi)$ is the projection of ϕ onto the set S (i.e., the point in S closest to ϕ). Applying this property to our objective function (Eq. 12) and assuming we can interchange expectation and differentiation, the gradient of the objective function is:

$$\nabla_{\phi} E_T \left[\frac{1}{2} D(\phi, \mathcal{W}_T)^2 \right] = E_T \left[\nabla_{\phi} \frac{1}{2} D(\phi, \mathcal{W}_T)^2 \right] = E_T [\phi - P_{\mathcal{W}_T}(\phi)] \quad (13)$$

Here, $P_{\mathcal{W}_T}(\phi)$ represents the projection of the current meta-parameters ϕ onto the optimal parameter set $\mathcal{W}_T$ for task T . The vector $\phi - P_{\mathcal{W}_T}(\phi)$ points from the closest optimal point for task T towards the current meta-parameters ϕ .

The objective function (Eq. 12) can be minimized using stochastic gradient descent. Based on the gradient derived in Eq. (13), an ideal meta-update step for the parameters ϕ using a single sampled task T would be:

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} \frac{1}{2} D(\phi, \mathcal{W}_T)^2 = \phi - \alpha(\phi - P_{\mathcal{W}_T}(\phi)) = (1 - \alpha)\phi + \alpha P_{\mathcal{W}_T}(\phi) \quad (14)$$

Here, α is the meta-learning rate (or outer loop learning rate). Each execution of such an update, typically based on a batch of tasks,

constitutes one meta-training iteration or "episode". This process of updating meta-parameters based on task adaptation results is termed episodic learning.

However, in practical applications, directly computing the projection $P_{\mathcal{W}_T}(\phi)$ is generally intractable because it requires finding the exact minimum of the task-specific loss function L_T , which itself is an optimization problem. Therefore, we approximate this projection using a fixed number of gradient descent steps performed on the task's loss function L_T , starting from the current meta-parameters ϕ . This task-specific adaptation process is often referred to as the "inner loop".

Let $W_T^*(\phi)$ be the parameters after running k steps of gradient descent on the loss function L_T starting from the initialization parameters ϕ . That is:

$$W_T^*(\phi) = \phi_k \quad (15)$$

Among them, ϕ_k is the parameter after k gradient descents, defined as follows:

$$\phi_{t+1} = \phi_t - \eta \nabla_{\phi_t} L_T(\phi_t), \text{ where } \phi_0 = \phi \quad (16)$$

Expanding this recursion gives the explicit formula for ϕ_k :

$$\phi_k = \phi - \eta \sum_{t=0}^{k-1} \nabla_{\phi_t} L_T(\phi_t) \quad (17)$$

Where η is the inner loop learning rate, controlling the step size during task adaptation. $L_T(\phi_t)$ is the loss of task T computed using parameters ϕ_t .

Now, we substitute our adaptive weighted loss function (10) for L_T . The gradient term in Eq. (17) becomes:

$$\phi_k = \phi - \eta \sum_{t=0}^{k-1} \nabla_{\phi_t} \frac{1}{N} \sum_{i=1}^N \left(\frac{\max(\text{freq}(y))}{\text{freq}(y_i) + \lambda} \right)^\beta \cdot (f_{\phi_t}(x_i) - y_i)^2 \quad (18)$$

Here, $f_{\phi_t}(x_i)$ represents the prediction of the meta-learner f with parameters ϕ_t for input x_i . The frequencies $\text{freq}(y_i)$ and the maximum frequency are computed based on the labels y_i within the batch.

Substituting the gradient expression from Eq. (18) into Eq. (17), we obtain the formula for the adapted parameters ϕ_k , which serves as our approximation $W_T^*(\phi)$:

$$W_T^*(\phi) = \phi - \eta \sum_{t=0}^{k-1} \nabla_{\phi_t} \frac{1}{N} \sum_{i=1}^N \left(\frac{\max(\text{freq}(y))}{\text{freq}(y_i) + \lambda} \right)^\beta \cdot (f_{\phi_t}(x_i) - y_i)^2 \quad (19)$$

This equation defines the parameters obtained after k steps of inner-loop adaptation using the weighted loss, starting from ϕ .

Now, we use this approximation $W_T^*(\phi)$ in place of the true projection $P_{\mathcal{W}_T}(\phi)$ in the ideal meta-update rule (Eq. 14). This yields the practical meta-update rule for FOMLN:

$$\phi \leftarrow (1 - \alpha)\phi + \alpha W_T^*(\phi) \quad (20)$$

Finally, substituting the full expression for $W_T^*(\phi)$ from (19) into (20), we arrive at the final parameter update formula used in our FOMLN strategy:

$$\phi \leftarrow (1 - \alpha)\phi + \alpha \left[\phi - \eta \sum_{t=0}^{k-1} \nabla_{\phi_t} \frac{1}{N} \sum_{i=1}^N \left(\frac{\max(\text{freq}(y))}{\text{freq}(y_i) + \lambda} \right)^\beta \cdot (f_{\phi_t}(x_i) - y_i)^2 \right] \quad (21)$$

Based on the above derivation, we can obtain the update rule of the first order episodic meta training strategy, as shown in **Algorithm**: First order episodic meta training strategy.

Algorithm. First-Order Episodic Meta-Training Strategy

$\alpha(W_T^*(\phi) - \phi)$. Thus, the core direction is determined by $W_T^*(\phi) - \phi$, which is equivalent to $\phi_k - \phi$. Let L_T represent the loss function for a

Require: Meta-task set $\mathcal{T} = \{T_1, T_2, \dots\}$; Meta-learning rate α , Inner loop learning rate η ; inner steps k ; Meta-batch size m ; Number of samples per task n (K -shot size for task T_j); Adaptive loss exponent β ; Smoothing factor λ .

1: **Initialize:** Meta-parameters ϕ randomly

2: **for** each training epoch **do**

3: **for** each meta-training episode **do**

4: // — Outer Loop: Sample Tasks —

5: Sample a meta-batch of m tasks $\mathcal{T}' = \{T_1, T_2, \dots, T_m\}$ from $\mathcal{T}$

6: **for** $j = 1$ to m **do** ▷ Process each task in the meta-batch

7: // — Inner Loop: Task Adaptation for Task T_j —

8: Sample task data $D_j = \{(x_i, y_i)\}_{i=1}^n$ from task T_j

9: Initialize base learner parameters for task T_j : $\phi'_j \leftarrow \phi$

10: Initialize an optimizer for the inner loop (e.g., Adam) with learning rate η for parameters ϕ'_j :

11: $Optimizer_j \leftarrow AdamOptimizer(params = \phi'_j, lr = \eta)$

12: **for** $step = 1$ to k **do** ▷ Perform k inner gradient steps

13: // Calculate adaptive weighted loss on task data D_j using current ϕ'_j

14: Calculate frequencies $freq(y_i)$ for all y_i in D_j based on counts $N_{y=y_i}$:

15: $freq(y_i) = \frac{N_{y=y_i}}{n}$

16: $Max_freq \leftarrow \max_{y \in \{y_1, y_2, \dots, y_n\}} \{freq(y)\}$

17: $Loss_j = \frac{1}{n} \sum_{i=1}^n \left(\frac{Max_freq}{freq(y_i) + \lambda} \right)^\beta \cdot (f_{\phi'_j}(x_i) - y_i)^2$ ▷ Using adaptive weighted MSE

18: // Compute gradient of the loss w.r.t. base learner parameters ϕ'_j

19: $g_j \leftarrow \nabla_{\phi'_j} Loss_j$

20: // Update base learner parameters using the optimizer

21: $\phi'_j \leftarrow Optimizer_j.step(g_j)$ ▷ Updates ϕ'_j based on g_j

22: **end for** ▷ End of inner loop steps for task T_j

23: // — Outer Loop: Meta-Parameter Update —

24: // Update meta-parameters ϕ towards the adapted parameters ϕ'_j from task T_j

25: $\phi \leftarrow (1 - \alpha)\phi + \alpha\phi'_j$ ▷ Meta-update

26: **end for** ▷ End loop over tasks in the meta-batch

27: **end for** ▷ End episodes

28: **end for** ▷ End epochs

29: **Output:** updated meta-parameters ϕ

3.3.2. Analysis of the meta-update

While the first order episodic meta training strategy is motivated by approximating the projection onto the task-optimal parameter set, its properties can be further elucidated through a theoretical analysis. We follow the methodology presented in [21] by performing a Taylor series expansion of the inner loop update.

Our goal is to analyze the expected meta-update direction over the distribution of tasks T . The meta-update can be written as $\phi_{new} \leftarrow \phi +$

specific task T , η be the inner loop learning rate, and k be the number of inner gradient steps. The parameters evolve from $\phi_0 = \phi$ to ϕ_k according to Eq. (16). We define the gradient of the task loss at the initial meta-parameters ϕ as $\bar{g} = \nabla_{\phi} L_T(\phi)$, and the Hessian matrix (matrix of second partial derivatives) of the task loss at ϕ as $H = \nabla_{\phi}^2 L_T(\phi)$. The gradient at step t is $g_t = \nabla_{\phi_t} L_T(\phi_t)$.

The total change in parameters after k inner steps is the sum of the individual steps:

$$\phi_k - \phi = \sum_{t=0}^{k-1} (\phi_{t+1} - \phi_t) = \sum_{t=0}^{k-1} (-\eta g_t) \quad (22)$$

To approximate this sum, we expand the gradient g_t around the

initial point ϕ using Taylor expansion. This expansion involves the gradient $\bar{g}$ and the Hessian H (both evaluated at ϕ), yielding the approximation:

$$g_t \approx \bar{g} + H(\phi_t - \phi) \quad (23)$$

We also approximate the trajectory of the parameters within the inner loop using a first-order estimate:

$$\phi_t - \phi = \sum_{j=0}^{t-1} (\phi_{j+1} - \phi_j) = -\eta \sum_{j=0}^{t-1} g_j \approx -t\eta \bar{g} \quad (24)$$

Substituting (24) into (23) gives:

$$g_t \approx \bar{g} + H(-t\eta \bar{g}) = (I - t\eta H)\bar{g} \quad (25)$$

This allows us to approximate the total parameter change:

$$\phi_k - \phi \approx \sum_{t=0}^{k-1} -\eta(I - t\eta H)\bar{g} \quad (26)$$

$$= -\eta \left(\sum_{t=0}^{k-1} I - \eta H \sum_{t=0}^{k-1} t \right) \bar{g} \quad (27)$$

$$= -\eta \left(kI - \eta H \frac{k(k-1)}{2} \right) \bar{g} \quad (28)$$

$$= -k\eta \bar{g} + \frac{k(k-1)}{2} \eta^2 H \bar{g} + O(\eta^3) \quad (29)$$

This expression approximates the vector pointing from the initial meta-parameters ϕ to the task-adapted parameters ϕ_k . The proposed update moves ϕ in this direction (scaled by α). To understand the average behavior, we consider the expectation over the distribution of tasks T :

$$E_T[\phi_k - \phi] \approx E_T \left[-k\eta \bar{g} + \frac{k(k-1)}{2} \eta^2 H \bar{g} \right] \quad (30)$$

$$= -k\eta E_T[\bar{g}] + \frac{k(k-1)}{2} \eta^2 E_T[H \bar{g}] \quad (31)$$

Eq. (31) provides insight into the objectives implicitly optimized by proposed method. The first term, $-k\eta E_T[\bar{g}]$, represents a scaled step along the average negative gradient across all tasks. This term drives the meta-parameters towards a point that minimizes the average task loss, similar to conventional multi-task or batch gradient descent. The second term, $(1/2)k(k-1)\eta^2 E_T[H \bar{g}]$, involves the expected product of the task Hessian and the task gradient. This term incorporates second-order information related to the curvature of the individual task loss landscapes. It implicitly modulates the meta-update based on how the task gradients change in their own vicinity, reflecting sensitivity to the loss function's geometry.

This analysis shows that our proposed method update $\alpha E_T[\phi_k - \phi]$ is not merely following the average gradient. Instead, it balances the objective of minimizing average task performance with an implicit consideration of second-order task characteristics captured by $E_T[H \bar{g}]$, confirming that this first order episodic meta training strategy belongs to the class of first-order meta-learning algorithms that leverage implicit second-order information.

Table 1
Hyperparameters for the Conformer block.

Hyperparameter Category	Parameter Name	Value
Multi-Head Self-Attention	Number of Heads (h)	8
	Dimension per Head (d_k, d_v)	64
	Attention Dropout Rate	0.2
FFM	FFM Dropout Rate	0.2
CM	Depthwise Convolution Kernel Size	31
	CM Dropout Rate	0.2

Table 2
Description of the C-MAPSS dataset.

Subset	Operational Condition	Fault Mode:	Training Set Units	Test Set Units
FD001	Single operating condition	Single fault mode	100	100
FD002	6 operating conditions	Single fault mode	260	259
FD003	Single operating condition	2 fault modes	100	100
FD004	6 operating conditions	2 fault modes	249	248

Table 3
Detailed description of sensors in the C-MAPSS dataset.

Number	Symbol	Description	Units
S1	T2	Temperature at the inlet of the fan	°R
S2	T24	Temperature at the LPC	°R
S3	T30	Temperature at the HPC	°R
S4	T50	Temperature at the LPT	°R
S5	P2	Fan inlet pressure	psia
S6	P15	Pressure within the bypass duct	psia
S7	P30	Pressure at the HPC outlet	psia
S8	Nf	Fan's physical rotational speed	rpm
S9	Nc	Core's physical rotational speed	rpm
S10	epr	Engine pressure	—
S11	Ps30	Static pressure at HPC outlet	psia
S12	phi	Fuel flow to Ps30 ratio	pps/psi
S13	NRf	Corrected rotational speed of fan	rpm
S14	NRc	Corrected rotational speed of core	rpm
S15	BPR	Ratio of bypass air to core air	—
S16	farB	Fuel-to-air ratio in the burner	—
S17	htBleed	Enthalpy of the bleed air	—
S18	Nf_dmd	Desired fan rotational speed	rpm
S19	PCNfR_d	Desired corrected fan rotational speed	rpm
S20	W31	Coolant bleed from the HPT	lbm/s
S21	W32	Coolant bleed from the LPT	lbm/s

4. Experiment

In this section, two case studies were conducted to verify the effectiveness of the proposed FOMLN in the scenario of few-shot RUL prediction: one using the public C-MAPSS dataset, and the other was conducted on the RUL prediction of slurry pumps in real industrial scenarios.

The model hyperparameters are set as follows: For the meta-learner, the encoder parameters include setting the kernel size of the 1D convolution layer to 10, the stride to 1, the number of kernels to 10, and the number of Conformer blocks to 1. Hyperparameters for the Conformer block is shown in Table 1.

The training parameters of the episodic meta training strategy include the inner layer loop learning rate of 0.001, the outer layer loop learning rate of 0.1, the number of inner loops of 10, and the number of outer loops of 50.

4.1. Case study 1: C-MAPSS dataset

4.1.1. Dataset description

The C-MAPSS dataset [40] is a benchmark dataset developed by NASA for RUL prediction in PHM. The dataset is obtained by simulating and studying the degradation process of the engine under different operating conditions. It contains the behavior data of simulated turbofan engines under various operating scenarios and conditions.

The overall description of the C-MAPSS dataset is shown in Table 2. It is divided into four subsets: FD001, FD002, FD003, and FD004. Each subset contains 21 types of sensor data (the specific information of each sensor is shown in Table 3). These subsets differ in terms of operating conditions and failure modes. In terms of dataset size, the training set

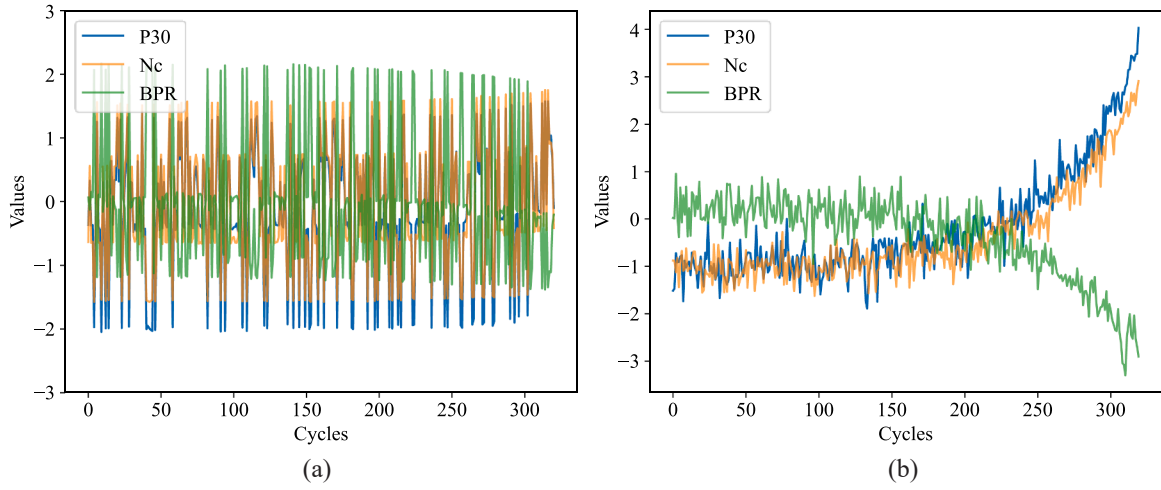


Fig. 5. Visualization of some sensor data of unit #1 on dataset FD004. (a). After GS, (b). After CS.

and test set of FD001 and FD003 each contain 100 units, the training set and test set of FD002 contain 260 and 259 units, respectively, and the training set and test set of FD004 contain 248 and 249 units, respectively.

4.1.2. Experimental design

We train and adapt the proposed method in a 15-shot scenario, and select the data of sensors No. 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14, 15, 17, 20, and 21 as features for analysis. These sensors cover key parameters such as temperature, pressure, speed, and flow, allowing the model to comprehensively capture the operating conditions of the engine and effectively evaluate its overall degradation degree. Since the FD002 and FD004 datasets contain multiple operating conditions, even samples with the same degradation degree will produce different eigenvalues due to different operating conditions, which is not conducive to the consistent learning of the model. Therefore, we follow the settings in

references [41] and [42] and perform operating condition-based standardization (CS) on the original data. Unlike global standardization (GS), CS standardizes data under different operating conditions separately. This can eliminate the influence of operating conditions on eigenvalues and make the features more consistent and comparable.

Fig. 5 shows the changes in some sensor values of Unit 1 engine in the FD004 data set after GS and CS processing. It can be seen that the data processed by GS does not show obvious degradation trends. Using this kind of data to train the model will make it difficult for the model to capture key degradation information, while CS overcomes this shortcoming. The eigenvalues after CS processing have an obvious degradation trend and thus have better discrimination and prediction capabilities.

We used a sliding window method to process the sensor data. The length of the sliding window is set to 30-time units, so the shape of each sample is finally adjusted to (15,30), where 15 represents the number of

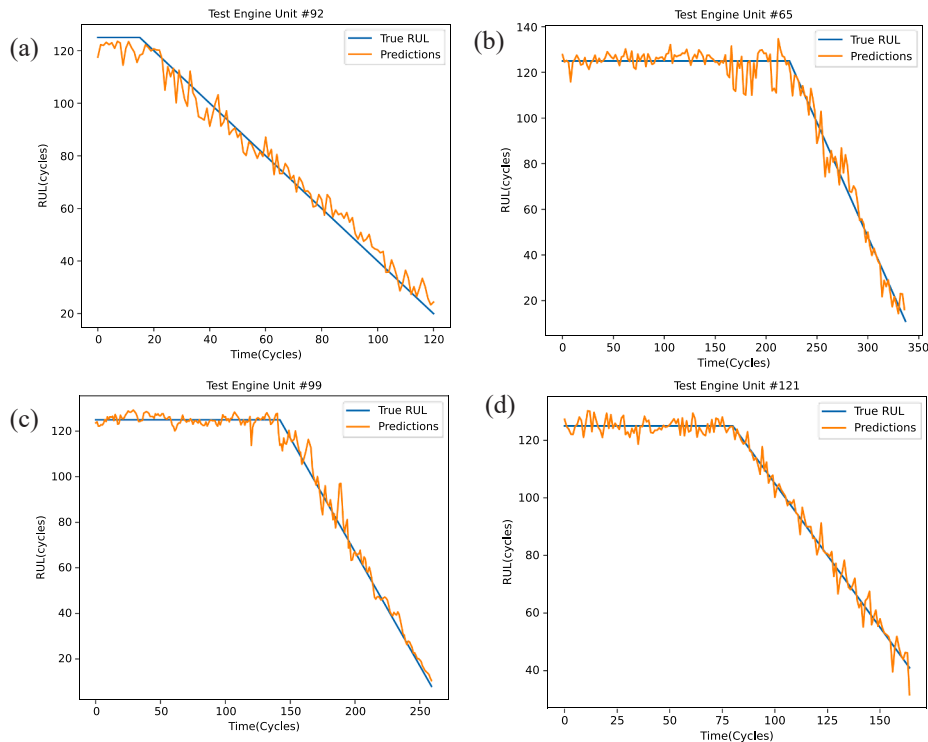


Fig. 6. Prediction results of the proposed method for a single unit under different tasks. (a)-(d) are engines randomly selected on FD001-FD004, respectively.

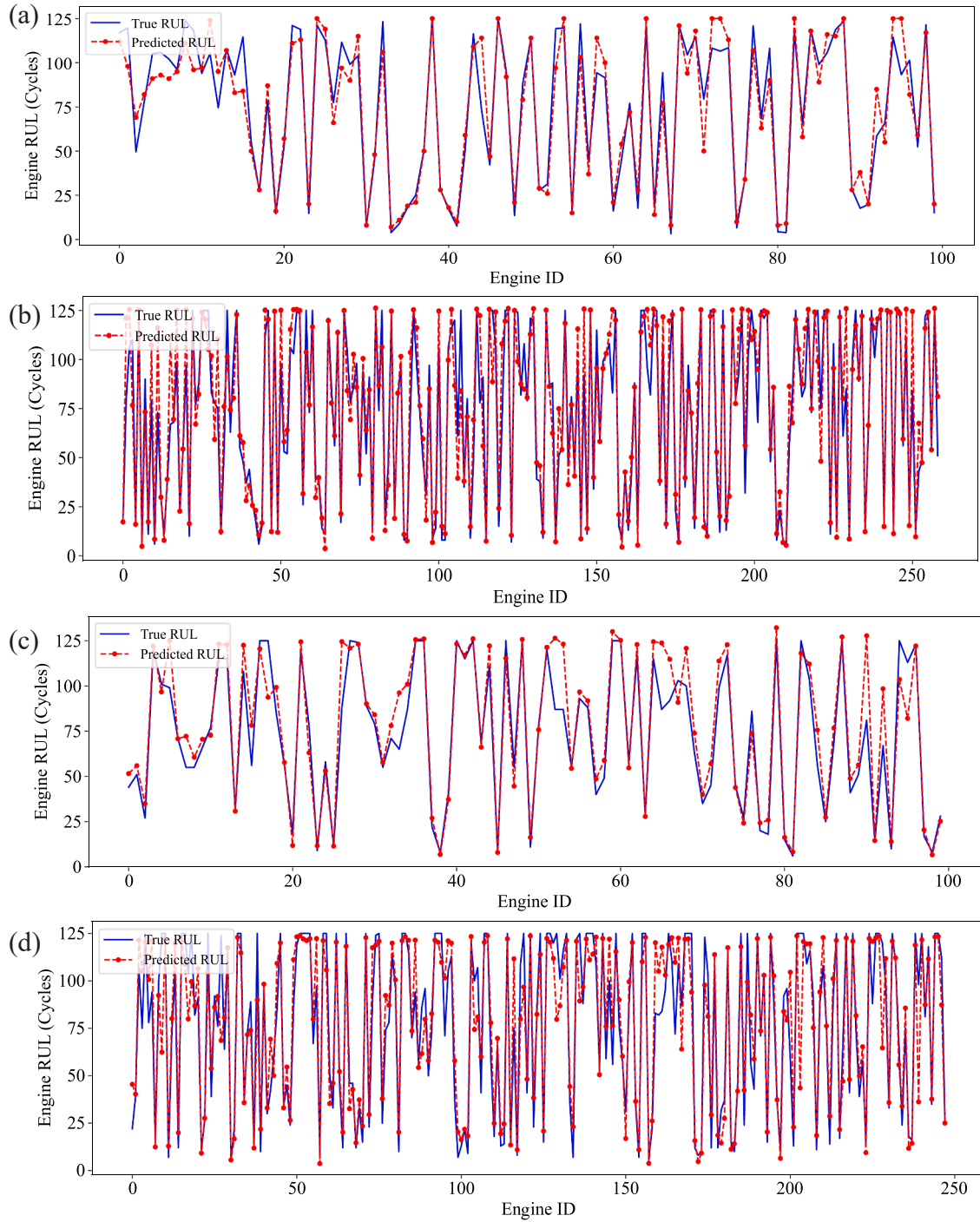


Fig. 7. Engine RUL prediction results. Figures (a) to (d) represent engines FD001 to FD004, respectively.

selected sensors and 30 represents the time window length.

When processing the C-MAPSS dataset for RUL prediction, we adopted the labeling strategy outlined in [43], capping the maximum RUL value for each unit at 125 cycles. This approach is based on the observation that equipment in a healthy state typically does not exhibit obvious signs of degradation. During this phase, the sensor data remain relatively stable with minimal variation. Assigning different RUL labels during this healthy period could lead the model to learn irrelevant or even misleading degradation patterns, compromising its predictive performance.

For the RUL prediction results of different models, we used the root

mean square value (RMSE) as an evaluation indicator to measure the difference between the predicted value and the true value, so as to compare the performance of different models. RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2} \quad (32)$$

In addition, we also used SCORE as another criterion for evaluating RUL prediction performance, which is defined as follows:

Table 4

Comparison of RMSE of the proposed method with recently published work.

Model	FD001	FD002	FD003	FD004
MS-DCNN [44]	11.44	19.35	11.67	22.22
LSTM-MLSA [45]	11.57	14.02	12.13	17.21
KGHM [3]	13.18	13.25	13.54	19.96
alarm-based CNN [46]	14.35	17.98	12.05	15.15
Semi-supervised [47]	12.56	22.73	12.10	22.66
VAE+RNN [41]	15.81	24.12	14.88	26.54
RVE [41]	13.42	14.92	12.51	16.37
ED-LSTM [48]	9.14	18.17	11.96	18.51
MSCCAN [49]	11.43	18.92	11.72	21.79
FedCov [50]	25.97	30.35	25.94	30.97
FOMLN(Ours)	10.43	13.93	11.61	15.94

Table 5

Comparison of SCORE of the proposed method with published work.

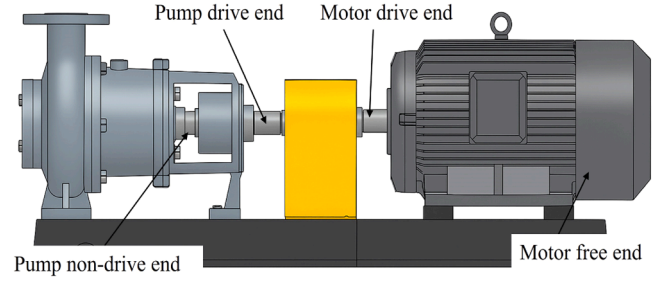
Model	FD001	FD002	FD003	FD004
MS-DCNN [44]	196.22	3747	241.89	4844
LSTM-MLSA [45]	252.86	899.18	370.39	1558.48
KGHM [3]	250.99	1131.03	333.44	3356.10
alarm-based CNN [46]	/	/	/	/
Semi-supervised [47]	231	3366	251	2840
VAE+RNN [41]	326	4183	722	5634
RVE [41]	323.82	1379.17	256.36	1845.99
ED-LSTM [48]	53	1693	238	2160
MSCCAN [49]	193.35	1468.84	293.51	2834.12
FedCov [50]	293.33	1238.73	318.85	1321.76
FOMLN (Ours)	212.01	1181.73	251.17	1749.66

$$SCORE = \begin{cases} \sum_{i=1}^n (e^{\frac{\hat{y}_i - y_i}{13}} - 1), & \text{if } \hat{y}_i < y_i \\ \sum_{i=1}^n (e^{\frac{\hat{y}_i - y_i}{10}} - 1), & \text{if } \hat{y}_i \geq y_i \end{cases} \quad (33)$$

4.1.3. Few-shot same-domain prediction

Compared with cross-domain tasks, domain-invariant tasks are generally considered to be relatively easier. This is because the model training and test data come from the same distribution, that is, the training set and test set come from the same C-MAPSS sub-dataset. Therefore, the model does not need to learn how to generalize from one domain to another.

Fig. 6 presents the RUL prediction results of the proposed method for a randomly selected unit across different tasks. The results indicate that: (1) The model demonstrates accurate RUL predictions across various units. For instance, on Unit #92 of FD001 and Unit #99 of FD003, the predicted values closely align with the actual RUL throughout the life-cycle, particularly in the later stages near failure, showcasing strong predictive capability. (2) On more complex cases like Unit #121 of

**Fig. 8.** Schematic diagram of the industrial slurry pump.**Table 6**

Comparison of RMSE values between the proposed method and published work.

Source	Target	LSTM-DANN [16]	DIDRLSTM [51]	MAML based [30]	CADA [52]	VLSTM-LWSAN [53]	FOMLN (Ours)
FD001	FD002	48.6	29.40	/	19.52	20.3	15.89
FD001	FD003	45.9	20.90	44.02	39.58	20.0	14.34
FD001	FD004	43.8	34.40	/	31.23	24.2	18.16
FD002	FD001	28.1	16.10	/	13.88	13.3	11.92
FD002	FD003	37.5	21.10	/	33.53	18.3	14.82
FD002	FD004	31.8	34.00	/	33.71	21.5	18.55
FD003	FD001	31.7	18.80	/	19.54	15.2	13.23
FD003	FD002	44.6	29.40	/	19.33	26.0	18.81
FD003	FD004	47.9	32.30	/	20.61	25.4	18.99
FD004	FD001	31.5	21.30	/	20.10	21.2	12.95
FD004	FD002	24.9	28.90	/	18.50	22.2	17.97
FD004	FD003	27.8	16.90	/	14.49	14.6	13.44

Table 7

Comparison of SCORE values between the proposed method and published work.

Source	Target	LSTM-DANN [16]	DIDRLSTM [51]	MAML based [30]	CADA [52]	VLSTM-LWSAN [53]	FOMLN (Ours)
FD001	FD002	93,841	4822	/	2122	/	1658.52
FD001	FD003	27,005	470	/	8415	/	849.77
FD001	FD004	57,044	5538	/	11577	/	1808.78
FD002	FD001	8411	213	/	351	/	327.15
FD002	FD003	17,406	474	/	5213	/	661.71
FD002	FD004	66,305	5471	/	15106	/	3645.53
FD003	FD001	5113	442	/	1451	/	603.24
FD003	FD002	37,297	4127	/	5257	/	4139.54
FD003	FD004	141,117	4123	/	3219	/	7351.94
FD004	FD001	7586	553	/	1840	/	426.88
FD004	FD002	17,001	3483	/	4460	/	4698.18
FD004	FD003	5941	249	/	682	/	602.56

Table 8
Pump operation conditions.

Parameter name	Working condition 1	Working condition 2
Operating power (kw)	137	178
Running speed (rpm)	800	980
Lift (m)	30	30
Nominal diameter of inlet and outlet (mm)	150	150
operation hours (h)	1200 +	1200 +

FD004, the predictions progressively converge with the actual RUL over time. These results confirm the model's effectiveness in predicting RUL and its adaptability to varying operational conditions and degradation stages.

Fig. 7 shows the overall performance of FOMLN in predicting the RUL of all units on all subsets of the C-MAPSS dataset. As can be seen from the figure, the predicted points of FOMLN in most cases are close to the true RUL, indicating its effectiveness.

Tables 4 and 5 show the comparison between the proposed method and the recently published work. From the tables, we can see that FOMLN outperforms most models on FD003 and FD004, showing its superiority in the task of RUL prediction. On the other datasets, although FOMLN's performance is not optimal, its RMSE and SCORE values are still close to the optimal model, showing its good robustness. In general, FOMLN's performance is as good as the SOTA model when dealing with complex RUL prediction problems, and it can maintain stable high performance in different test scenarios.

4.1.4. Few-shot cross-domain prediction

Compared to domain invariant tasks, few-shot cross-domain tasks are more challenging because the model must learn to extract knowledge from one domain's data and apply it to a different distribution in another domain. This requires the model not only to capture important features within the domain but also to possess a certain degree of domain invariance to adapt to the data distribution of the new domain. Tables 6 and 7 show the results of the few-shot cross-domain tasks compared with published works. In the tables, Source represents the source domain dataset, Target represents the target domain dataset, the model is trained on the source domain, and tested on the target domain's test set. From the tables, we can see that: (1) The proposed method demonstrates excellent performance in multiple tasks. Particularly in few-shot cross-domain tasks such as FD004 to FD001 and FD002 to FD001, it outperforms other methods. The proposed method achieves the lowest RMSE values in all scenarios, indicating its strong predictive ability in few-shot cross-domain learning scenarios. (2) Although LSTM-DANN[16], DIDRLSTM[51], CADA[52] and VLSTM-LWSAN[53] methods have excellent performance in multiple tasks and achieved lower SCORE, they use a large number of unlabeled samples in the target domain for domain adaptation, which means that they have more

information to learn and adapt, which may be a limitation in practical applications because a large number of unlabeled samples in the target domain may be difficult to obtain. The proposed method was trained and domain adapted in a 15-shot scenario, that is, FOMLN only used 15 samples in the target domain and achieved performance similar to or even better than the SOTA model based on transfer learning, which shows the superiority of meta-learning-based models in the face of domain shift problems in the context of scarce samples. (3) Among meta-learning-based models, FOMLN's performance is better than MAML-based model[30]. This demonstrates that the proposed method also has advantages when compared with models of the same type.

4.2. Case study 2: RUL prediction using real-world data

4.2.1. Experimental design

To establish the practical application value of the proposed method in this study, we conducted experiments on few-shot RUL prediction of slurry pumps at an industrial site. Due to confidentiality agreements, actual images of the industrial site slurry pumps cannot be disclosed; therefore, schematic diagrams are used to represent the equipment used in the experiments, as shown in Fig. 8. At the experimental site, we monitored the operation data of the slurry pump under two working conditions, each consistent with the information listed in Table 8.

During the experiment, a vibration sensor was installed on the non-drive end of the pump to collect vibration signals throughout the pump's entire operational life. The sensor had a sampling frequency of 12,800 Hz, and data was recorded every half hour. Each recording lasted 2.56 s, resulting in 32,768 data points per recording ($12,800 \text{ Hz} \times 2.56 \text{ s}$). Over the pump's operational lifetime, exceeding 1200 h for each condition, approximately 2400 raw time-series recordings were collected. The operating conditions listed in Table 8 were maintained relatively stable during each phase. However, minor fluctuations inherent to the industrial process were present. The ground truth RUL labels was determined by the maintenance log. Specifically, it was defined by the point in time when the pump was shut down due to significantly degraded performance attributed to casing wear. This casing wear was confirmed by subsequent visual inspection (as shown in Fig. 9), which revealed severe wear due to mineral fragment impact, the transport of corrosive media, and variations in pump operating conditions.

4.2.2. Data preprocessing

We selected a variety of features of the vibration signal from the time domain to the frequency domain for analysis. The details are shown in Table 9. By calculating the Pearson correlation coefficient, we analyzed the correlation between the features in the table and the running time, and selected the 15 features with the strongest correlation with the running time as the feature set for model training. The Pearson correlation coefficient is defined as :

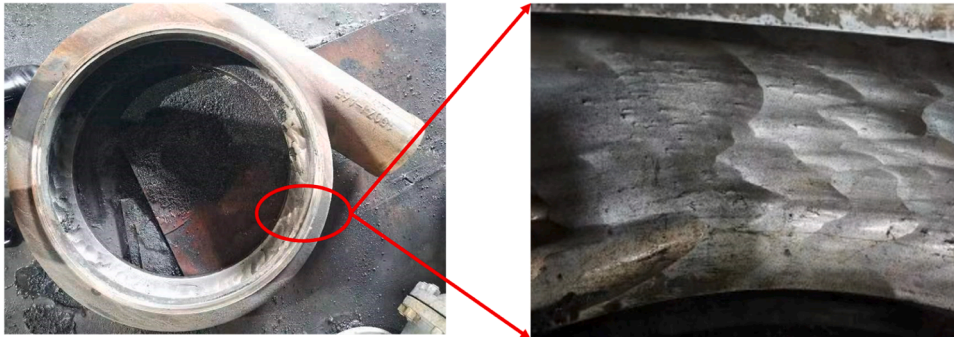


Fig. 9. Wear condition of pump casing.

Table 9

The selected features.

Feature name	Definition	Description
Mean	$\frac{1}{N} \sum_{i=1}^N x_i$	The average level of signal values, used to reflect the center point of the signal.
STD	$\sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \text{mean})^2}$	A measure of the volatility or dispersion of a signal's value.
RMS	$\sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$	The overall energy level of the signal
Max	$\max(x_i)$	The maximum amplitude of the signal.
Min	$\min(x_i)$	The minimum amplitude of the signal.
Peak to peak	$\max(x_i) - \min(x_i)$	The range of the signal reflects the amplitude range of the signal
Kurt	$\frac{N(N+1)}{(N-1)(N-2)(N-3)} \sum_{i=1}^N \left(\frac{x_i - \text{mean}}{\text{std}} \right)^4 - \frac{3(N-1)^2}{(N-2)(N-3)}$	Describes the kurtosis of a signal distribution, with high kurtosis indicating a higher frequency of outliers.
Skewness	$\frac{1}{N} \sum_{i=1}^N (x_i - \text{mean})^3$ $\left(\frac{1}{N} \sum_{i=1}^N (x_i - \text{mean})^2 \right)^{\frac{3}{2}}$	Describes the degree of asymmetry of the signal distribution.
Crest factor	$\frac{\max(x_i)}{\sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}}$	The ratio of peak to rms values, used to identify extreme peaks in a signal.
Shape factor	$\frac{\sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}}{\frac{1}{N} \sum_{i=1}^N x_i}$	The ratio of the root mean square value to the mean absolute value.
Clearance factor	$\frac{\max(x_i)}{\left \frac{1}{N} \sum_{i=1}^N x_i \right }$	Reflects the ratio of signal peak to average.
Impulse factor	$\frac{\max(x_i)}{\text{kurt}}$	The ratio of signal peak to kurtosis, a measure of signal impulse characteristics.
Peak frequency	$\text{freqs}[\arg\max(\text{PSD}(f))]$ where $\text{PSD}(f) = \frac{1}{N} \text{FFT}(x(f)) ^2$	Indicates the frequency point where the signal energy is most concentrated.
Total power	$\sum \text{PSD}(f)$	Reflects the total energy of the signal in the frequency domain.
Spectral centroid	$\frac{\sum (\text{freqs} \times \text{PSD}(f))}{\sum \text{psd}}$	The "center of gravity" of the spectrum reflects the balance of signal energy distribution.
Spectral Kurt	$\frac{\sum_{i=1}^N (\text{psd}_i - \overline{\text{psd}})^4 / N}{\left(\sum_{i=1}^N (\text{psd}_i - \overline{\text{psd}})^2 / N \right)^2}$	The kurtosis of the power spectral density. Where psd_i is the power spectral density value of the i th frequency point, $\overline{\text{psd}}$ is the average value of the power spectral density.
Spectral skew	$\frac{\sum_{i=1}^N (\text{psd}_i - \overline{\text{psd}})^3 / N}{\left(\sum_{i=1}^N (\text{psd}_i - \overline{\text{psd}})^2 / N \right)^{\frac{3}{2}}}$	Skewness of the power spectral density.
Stft magnitude	$\text{mean}(\text{STFT}(x))$	Amplitude average of short-time Fourier transform (STFT).
Energy of IMF _{<i>i</i>}	$\sum_{t=1}^T \text{IMF}_i(t)^2$	In empirical mode decomposition (EMD), the energy of each intrinsic mode function (IMF) component

$$r_{ab} = \frac{\sum_{i=1}^n (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^n (a_i - \bar{a})^2 \sum_{i=1}^n (b_i - \bar{b})^2}} \quad (34)$$

Here, a_i and b_i are the observed values of the feature and the corresponding running time respectively, $\bar{a}$ and $\bar{b}$ are their respective means, and n is the number of observations.

The visualization results of the 15 selected features are presented in Fig. 10, it can be observed that all features exhibit noticeable fluctuations, which are attributed to the continuous performance degradation caused by the harsh operating conditions of the slurry pump. Despite the severe fluctuations, most features show an overall upward trend over time, indicating that the equipment has been in a state of degradation since the beginning of its operation. Based on this observation, there is no need to use piecewise functions to describe the health state when setting RUL labels for the samples in this study. We employed the same data normalization method as in Case Study 1 and utilized a sliding window technique to expand the sample dimensions to (15, 30), in order to align with the input format of the model.

For comparison, we also constructed several benchmark models, including a Deep Convolutional Neural Network (DCNN) and a Transformer, structure of the DCNN is consistent with the one described in [54]. It processes the 2D input (time window $\times$ features) using primarily 1D convolutions along the time dimension. Four initial convolutional

layers (10 filters each, kernel size 10×1 , tanh activation, zero-padding) are stacked, followed by a fifth convolutional layer (1 filter, kernel size 3×1 , tanh activation) to combine the preceding feature maps. The resulting feature map is then flattened, passed through dropout with a rate of 0.5, followed by a fully connected layer (100 neurons, tanh activation), and finally connected to a single linear output neuron for RUL prediction. The Transformer baseline employed an encoder-only architecture follows [36], specifically adapted for the time-series regression task. It consisted of 2 stacked Transformer encoder layers Transformer encoder's structure, with the number of heads set to 4, the dimension set to 128, a dropout rate of 0.1, and 2 layers, using a linear layer as the regressor.

Based on them, DCNN-Transfer and Transformer-Transfer based on the fine-tuning transfer learning framework, as well as Meta-DCNN and Meta-Transformer trained using the first-order episodic meta-training strategy, are built to comprehensively evaluate the performance of the proposed method.

During the model training phase, we used the data from operating condition 1 as the source domain dataset, maintaining the same model parameters as in Case Study 1. For the target domain, using the data from operating condition 2, the model performed rapid adaptation to new tasks in a 10-shot scenario, then evaluated its performance using the remaining target domain samples. Finally, the model's performance

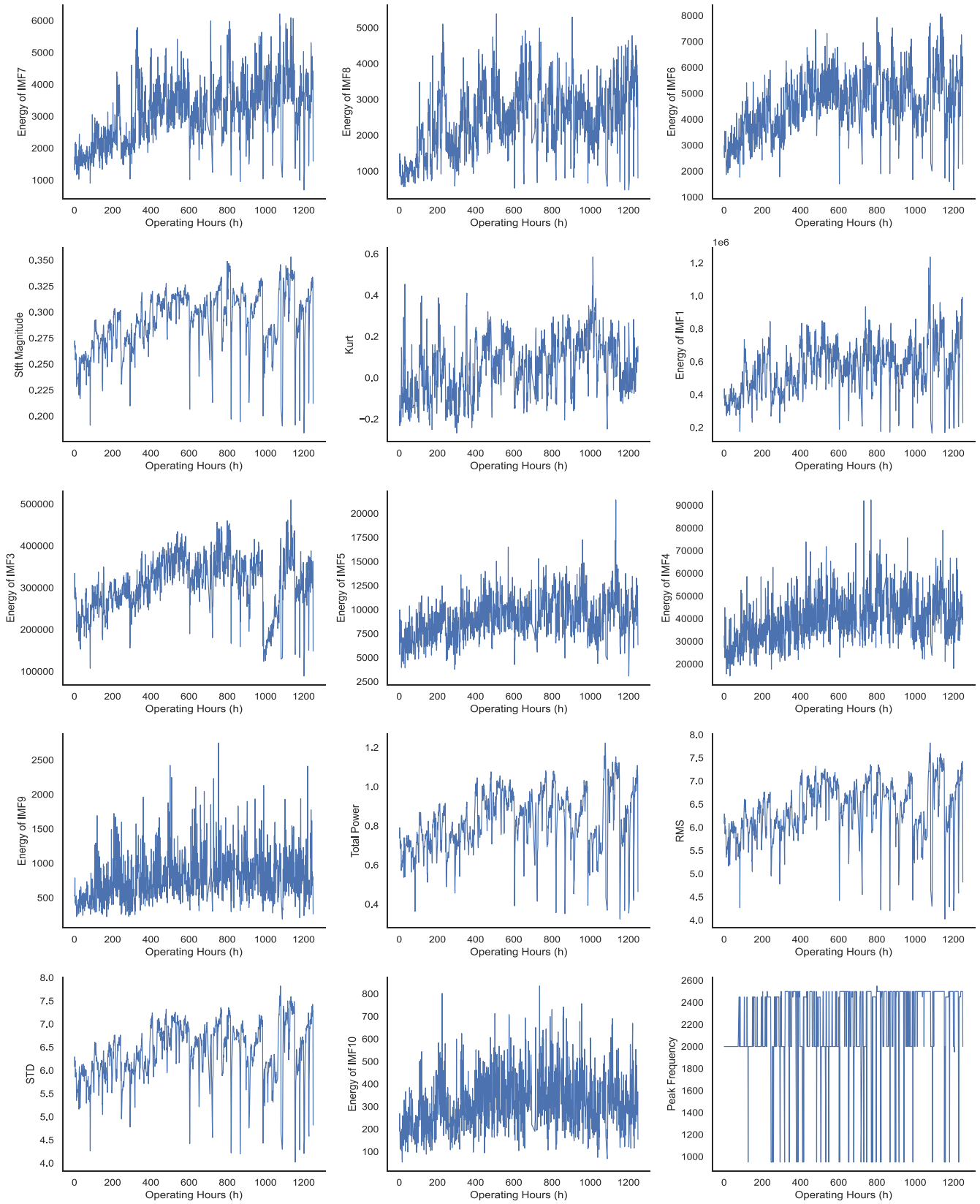


Fig. 10. Visualization of selected features.

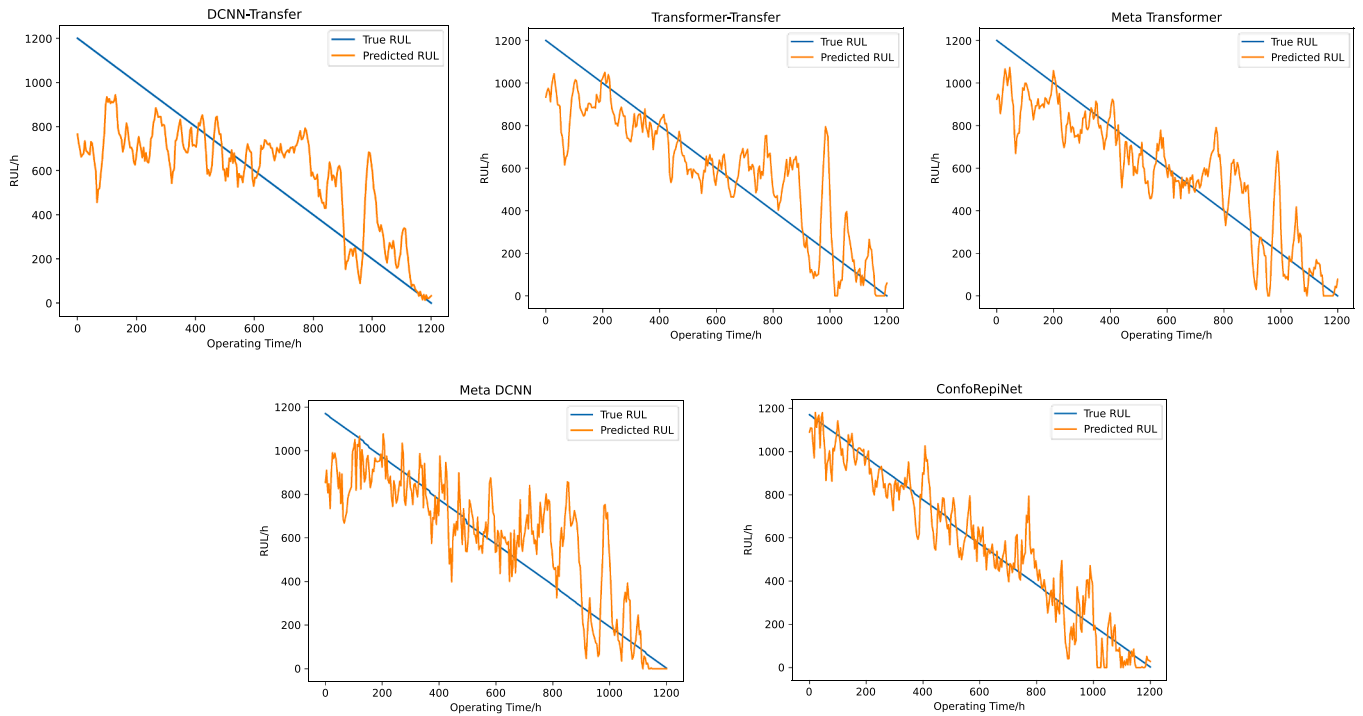


Fig. 11. Prediction results of different models.

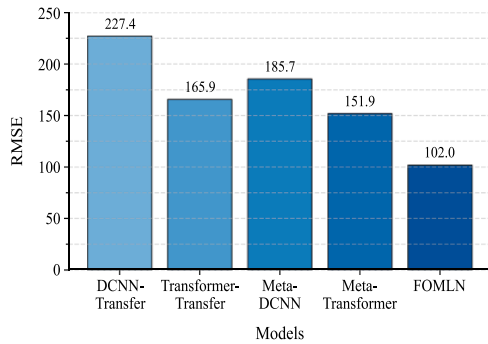


Fig. 12. RMSE of different models.

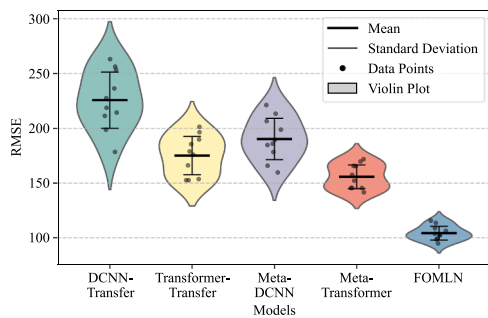


Fig. 13. Distribution of RMSE values over 10 repeated runs for different models on the industrial slurry pump few-shot cross-domain RUL prediction task.

was compared with several benchmark models—including DCNN-Transfer, Transformer-Transfer, Meta-DCNN, and Meta-Transformer—to highlight the advantages and applicability of the proposed method in real industrial applications.

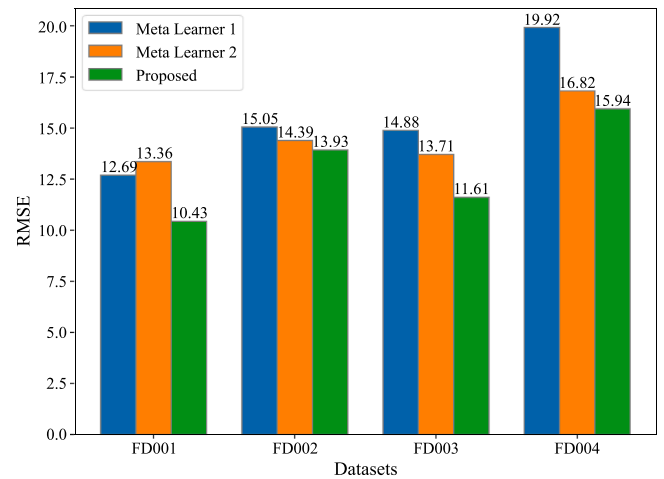


Fig. 14. Comparison of RMSE values for different meta learners.

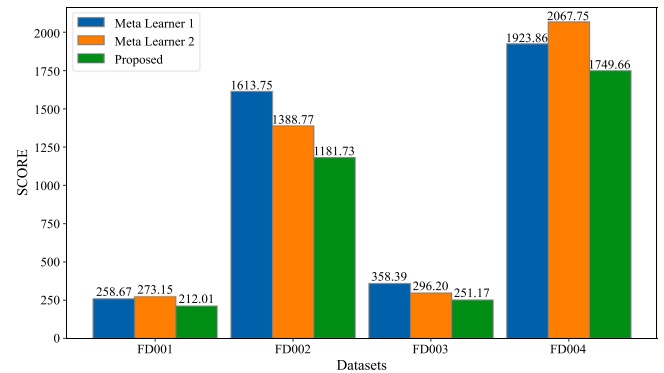


Fig. 15. Comparison of SCORE values for different meta learners.

4.2.3. Experimental results and analysis

Fig. 11 illustrates the match between the predicted RUL and the actual RUL for each model in the 10-shot learning cross-domain task. From the Fig. 11, it is evident that few-shot cross-domain RUL prediction in real industrial environments is more challenging compared to Case Study 1. In industrial scenarios, the monitored equipment's condition changes continuously with production tasks, and there is more noise and interference present in the industrial site, making the collected data less ideal than laboratory data. Fig. 11 shows that all models' predictions exhibit some degree of fluctuation. Notably, models using DCNN as the encoder tend to predict the RUL as the average of the total RUL (approximately 600 h). This is because convolution-based models can only focus on local information while ignoring global information, leading to a near-total loss of capability to capture degradation information of the equipment in complex and challenging industrial settings. Consequently, these models resort to predicting an average value to minimize training loss.

In contrast, FOMLN's performance stands out, with its predicted values closely aligning with the actual RUL values during the latter stages of operation, highlighting its ability to grasp task-relevant information and adapt to new environments. The RMSE comparison in Fig. 12 further underscores FOMLN's superiority: it achieved the lowest RMSE value of 102.03 among all models, reflecting its prediction accuracy. This performance demonstrates FOMLN's adaptability and prediction accuracy in handling few-shot cross-domain tasks within complex industrial environments. It showcases the model's accuracy and reliability in capturing equipment degradation information, emphasizing its practical applicability in real industrial settings.

Furthermore, to empirically evaluate the stability and robustness of the proposed FOMLN framework, we conducted 10 repeated experiments. Fig. 13 presents the distribution of the final RMSE results across these 10 runs using violin plots. As clearly depicted, FOMLN not only achieves the lowest average RMSE compared to all baseline methods but also exhibits the most concentrated distribution. The violin plot for FOMLN is notably narrower, and its standard deviation is considerably smaller than those of the competing models, especially compared to the DCNN-based approaches which show wide variability.

4.3. Ablation study

To better explore the impact of each component of the proposed FOMLN on the overall model performance, we designed several ablation experiments to systematically evaluate the contribution of each part of the model.

4.3.1. Meta-learner architecture

In this experiment, we constructed different meta-learner

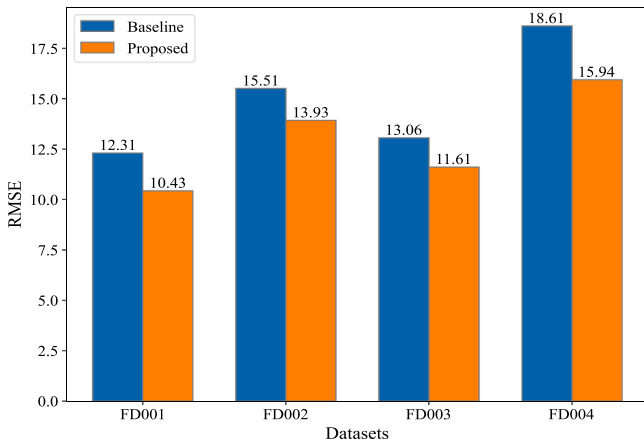


Fig. 16. Comparison of RMSE values for baseline and proposed models.

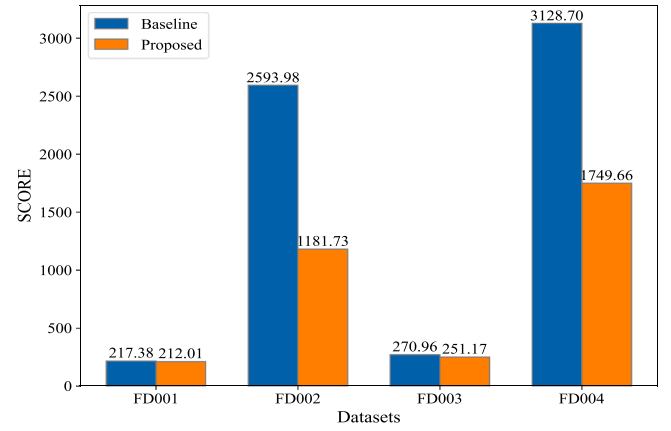


Fig. 17. Comparison of SCORE values for baseline and proposed models.

architectures and used the same meta-learning strategy for training and testing to discuss the effectiveness of the Conformer-based meta-learner in improving overall model performance. For comparison, we constructed a CNN-based meta-learner (referred to as Meta-Learner 1, with the structure consistent with [54]), a self-attention mechanism-based meta-learner (referred to as Meta-Learner 2, with the structure consistent with [36]), and the proposed meta-learner that combines the advantages of both. Experiments were conducted on four datasets under the same few-shot scenarios. The RMSE results are shown in Fig. 14, and the SCORE results are shown in Fig. 15.

From the figures, it can be observed that the proposed meta-learner achieved the lowest RMSE and SCORE values across all datasets, fully demonstrating the effectiveness of combining convolutional neural networks and self-attention mechanisms in few-shot RUL prediction tasks. By integrating both architectures, we achieved superior performance compared to using a single architecture, validating the superiority and applicability of this method in complex tasks.

4.3.2. Adaptive weighted loss

To investigate the impact of the proposed adaptive weighted loss function on the performance of the proposed method in few-shot scenarios, we constructed a model using the standard MSE loss function, referred to as Baseline. Except for the loss function, the structure and hyperparameter configuration of this model are identical to FOMLN. We conducted experiments on four datasets in case study 1 to test their performance. Figs. 16 and 17 respectively present the comparison of their RMSE and SCORE values.

From the figures, it can be observed that on the simpler FD001 and FD003 datasets, the performance difference between the proposed method and the Baseline is minimal. However, on the more variable FD002 and FD004 datasets, the proposed method significantly reduces the SCORE. Although the RMSE indicates that the average prediction error of both methods is relatively close, the Baseline tends to predict higher RUL values due to the higher frequency of high RUL samples seen during training, resulting in a higher SCORE. This outcome demonstrates the clear advantage of the adaptive weighted loss function in handling label imbalance, particularly in complex conditions and few-shot scenarios, where it can enhance the model's prediction accuracy and robustness.

4.3.3. Meta-Learning Strategies

To further verify the effectiveness of the first order episodic meta training strategy in the proposed method, we use the same meta-learner and the same hyperparameters to build baseline models based on traditional machine learning frameworks and different meta-learning frameworks, including standard MAML and the computationally cheaper first-order approximation FOMAML [20]. All methods were trained and evaluated on the C-MAPSS dataset under the same few-shot

Table 10
Performance comparison against meta-learning baselines.

Dataset	FD001		FD002		FD003		FD004	
	RMSE	SCORE	RMSE	SCORE	RMSE	SCORE	RMSE	SCORE
Baseline (Traditional ML)	13.35	254.25	15.46	2569.35	14.96	485.96	17.85	3365.24
Baseline (MAML)	12.30	265.54	14.74	2175.54	12.78	328.35	16.75	2944.25
Baseline (FOMAML)	13.23	245.36	15.43	1751.45	13.23	354.12	17.14	2841.25
Proposed	10.43	212.01	13.93	1181.73	11.61	251.17	15.94	1749.66

Table 11
Training time comparison against different meta learning strategies.

Meta-Learning Strategy	FD001 (Time in seconds)	FD002 (Time in seconds)	FD003 (Time in seconds)	FD004 (Time in seconds)
Baseline (MAML)	1845	3758	1724	3574
Baseline (FOMAML)	782	1532	744	1507
Proposed	614	1284	608	1216

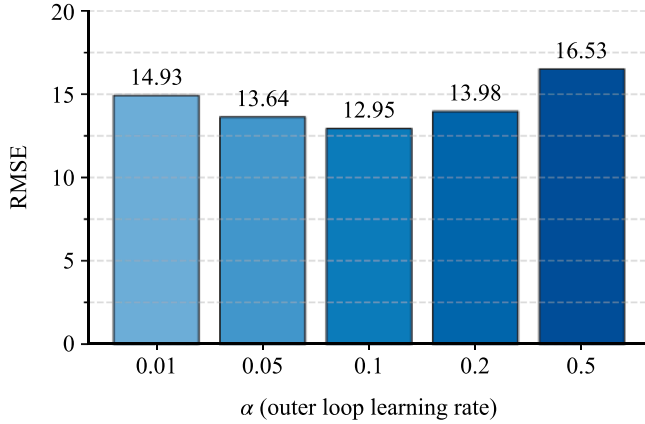


Fig. 18. Sensitivity of FOMLN performance to the outer loop learning rate α .

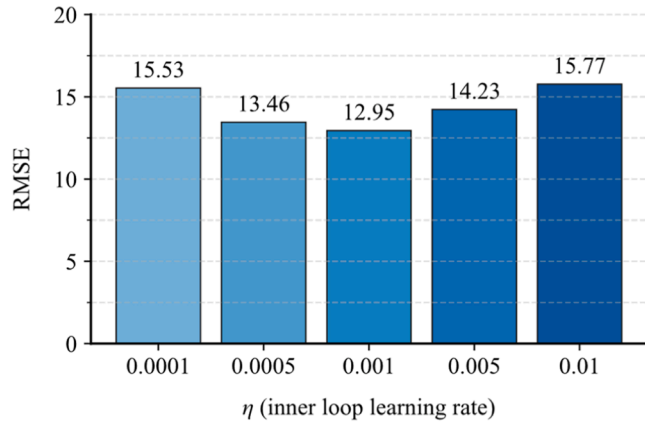


Fig. 19. Sensitivity of FOMLN performance to the inner loop learning rate η .

same-domain prediction setting described in Section 4.1.3. The experiments were performed on a system equipped with an NVIDIA GeForce RTX 3060 GPU and an Intel Core i5-12490F CPU using the PyTorch framework.

Results including RMSE, SCORE and total training time are shown in Table 10 and Table 11. It can be seen that the proposed meta training strategy consistently maintains the best performance on all four C-MAPSS datasets and obtains the lowest RMSE and SCORE values. In

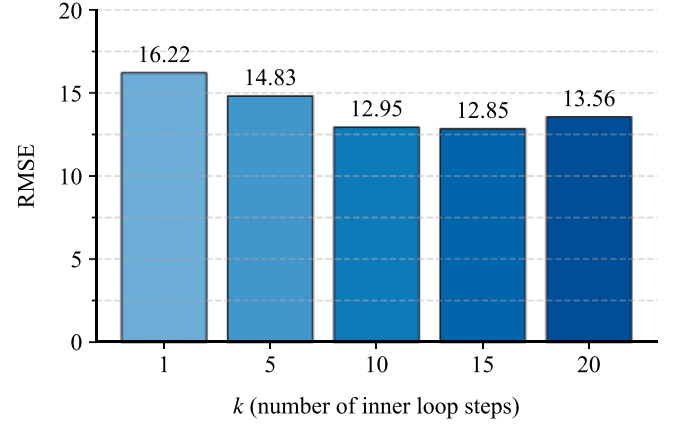


Fig. 20. Sensitivity of FOMLN performance to the number of inner loop steps k .

addition, standard MAML has the longest training time among all datasets, because it needs to compute second-order gradients to differentiate the inner-loop updates. FOMAML achieves improvements by using first-order approximations, but it still needs to backpropagate through the inner optimization path. The proposed method updates the meta-parameters by moving them towards the task-adaptive parameters without back-propagating through the adaptation process itself. Since differentiation through the inner steps is avoided, the computational cost of the meta-update is very low compared to MAML and FOMAML. Therefore, although the inner-loop computational cost of different methods is similar, the overall efficiency improvement brought by the streamlined meta-update makes our proposed method have the shortest training time on all datasets.

4.4. Sensitivity analysis

To thoroughly evaluate the proposed FOMLN framework and provide deeper insights into its behavior, we conducted a sensitivity anal-

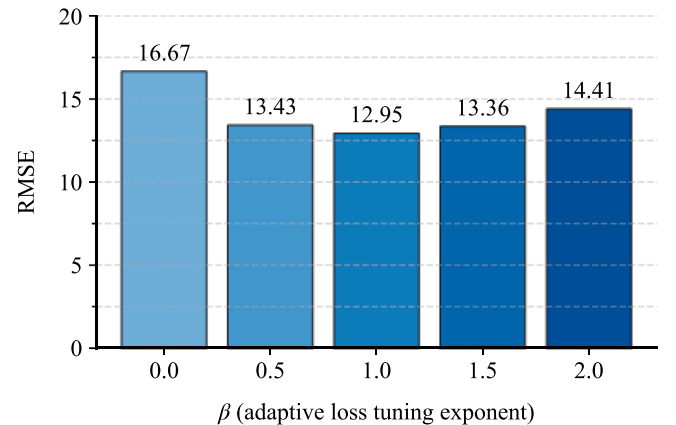


Fig. 21. Sensitivity of FOMLN performance (RMSE) to the adaptive loss tuning exponent β .

ysis on key parameters of the proposed FOMLN framework. We investigated the impact of the outer loop learning rate α , the inner loop learning rate η , the number of inner loop steps k , and the adaptive weighted loss tuning exponent β on the model's performance, aiming to provide insights into hyperparameter selection and their influence on convergence and stability.

The sensitivity analysis was performed on the challenging cross-domain task FD004 to FD001 under the 15-shot scenario, as this scenario involves complex operating conditions, multiple fault modes, and domain shift, making it representative for evaluating robustness. For each parameter tested, others were kept at their default values reported in Section 4 ($\alpha = 0.1$, $\eta = 0.001$, $k = 10$, $\beta = 1$), unless otherwise specified. Performance was evaluated using RMSE.

4.4.1. Learning rates α and η

The dual-loop learning rates, α (outer) and η (inner), are crucial for balancing meta-level generalization and task-specific adaptation. We varied α within the range [0.01, 0.05, 0.1, 0.2, 0.5] while keeping $\eta = 0.001$ and $k = 10$. Subsequently, we varied η within [0.0001, 0.0005, 0.001, 0.005, 0.01] while keeping $\alpha = 0.1$ and $k = 10$. The RMSE results are shown in Figs. 18 and 19.

As shown in Figs. 18 and 19, a very small α (e.g., 0.01) resulted in slow convergence of the meta-parameters, leading to suboptimal performance within the fixed number of outer loop epochs. Conversely, a large α (e.g., 0.5) caused instability in the meta-update, where the meta-parameters oscillated and failed to converge to a good initialization, resulting in poor performance. The value $\alpha = 0.1$ provided a good balance, enabling stable and efficient convergence towards a robust meta-initialization. And the inner loop learning rate dictates the speed and extent of adaptation within each task. A very small η (e.g., 0.0001) led to insufficient adaptation within the $k=10$ steps, meaning the base learners did not adequately fit their respective tasks, hindering the quality of the meta-gradient estimate. A large η (e.g., 0.01) caused the inner loop optimization to be unstable or overshoot the optimal parameters for the task, also degrading meta-learning performance. The value $\eta = 0.001$ allowed for effective task adaptation within the inner loop without causing instability.

4.4.2. Number of inner loop steps k

The number of inner loop gradient descent steps, k , determines how much adaptation occurs for each task before the meta-update. We varied k in the range [1, 5, 10, 15, 20] while keeping $\alpha = 0.1$ and $\eta = 0.001$. Results are shown in Fig. 20.

Fig. 20 indicates that performance generally improves as k increases from 1 to 10. With very few steps ($k=1$ or $k=5$), the base learners cannot sufficiently adapt to their specific tasks, resulting poor meta-update quality. As k increases to 10, the adaptation becomes more refined, leading to better meta-learning. However, increasing k further ($k=15$ or $k=20$) showed diminishing returns and even a slight increase in RMSE. This could be attributed to potential overfitting within the inner loop to the small number of samples (15-shot) in each task, which might slightly harm the generalization ability captured by the meta-parameters. Additionally, larger k greatly increases the computational cost of training. Therefore, $k=10$ was selected as it offered the best balance between adaptation quality, generalization performance, and computational efficiency.

4.4.3. Adaptive loss tuning exponent β

The adaptive weighted loss function includes a tuning exponent β that controls the strength of the weighting applied to address label imbalance. We investigated the impact of varying β in the range [0, 0.5, 1.0, 1.5, 2.0], where $\beta=0$ corresponds to the standard MSE loss (equivalent to the Baseline model in the ablation study).

The RMSE results are shown in Fig. 21, showing that setting $\beta = 0$ (standard MSE) results in the highest RMSE, as standard MSE has difficulty coping with unbalanced RUL values. As β increases to 0.5 and

1.0, the model performance improves. The best performance is achieved around $\beta = 1.0$. Further increasing β to 1.5 and 2.0 results in a slight decrease in performance. This suggests that too high a weighting (high β) may cause the model to be overly sensitive to minority class samples or potential noise therein, especially in the few-shot scenario. Therefore, we choose $\beta = 1$ as the optimal value.

5. Conclusion

This study proposes an innovative meta-learning model—FOMLN, specifically designed for RUL prediction tasks of rotating machinery in limited sample scenarios. FOMLN's meta-learner integrates convolution and self-attention mechanisms within the Conformer architecture. Using a first-order episodic meta-learning strategy, it performs dual-loop learning at multiple levels: the inner loop conducts fine-grained learning at the sample level, capturing task-specific details and updating parameters; the outer loop operates at the task level, gradually aligning the model parameters with the optimal parameter set for each meta-task. This approach enhances the model's generalization capability across tasks and its rapid adaptation to new tasks in few-shot scenarios.

Extensive experiments were conducted on various datasets to evaluate the performance of the proposed method. Detailed experiments using the C-MAPSS dataset confirmed the effectiveness of FOMLN in FSL scenarios. The results demonstrated not only FOMLN's superior performance across single fault modes and different operating conditions but also highlighted its enhanced prediction accuracy and reduced reliance on target domain sample sizes in few-shot cross-domain learning tasks, compared to other state-of-the-art models. In a critical industrial application involving the prediction of the RUL of slurry pumps in FSL scenarios, FOMLN greatly outperformed other benchmark models, including traditional transfer learning and meta-learning approaches.

CRedit authorship contribution statement

Yu Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation. **Shujie Liu:** Supervision, Investigation, Funding acquisition. **Shuai Lv:** Writing – review & editing, Writing – original draft, Visualization, Validation. **Gengshuo Liu:** Writing – original draft, Validation.

Declaration of Competing Interest

The authors declare that they have no financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The work is supported by the National Natural Science Foundation of China (No. 51975100) and the National Key Research and Development Program of China (No.2019YFB2004600)

Data availability

The authors do not have permission to share data.

References

- [1] J. Chen, D. Li, R. Huang, Z. Chen, W. Li, Aero-engine remaining useful life prediction method with self-adaptive multimodal data fusion and cluster-ensemble transfer regression, *Reliab. Eng. Syst. Saf.* 234 (2023).
- [2] L. Wang, H. Cao, H. Xu, H. Liu, A gated graph convolutional network with multi-sensor signals for remaining useful life prediction, *Knowl. Based Syst.* 252 (2022).

- [3] Y. Li, Y. Chen, Z. Hu, H. Zhang, Remaining useful life prediction of aero-engine enabled by fusing knowledge and deep learning models, *Reliab. Eng. Syst. Saf.* 229 (2023).
- [4] X. Li, J. Li, L. Zuo, L. Zhu, H.T. Shen, Domain Adaptive Remaining Useful Life Prediction With Transformer, *IEEE Trans. Instrum. Meas.* 71 (2022) 1–13.
- [5] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural comput* 9 (1997) 1735–1780.
- [6] J. Chung, C. Gulcehre, K. Cho, and Y.J. a p a Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, (2014).
- [7] P. Ding, M. Jia, X. Zhao, Meta deep learning based rotating machinery health prognostics toward few-shot prognostics, *Appl. Soft Comput.* 104 (2021).
- [8] J. Hu, L. Wu, Early Uncertainty Quantification Prediction of Lithium-ion Battery Remaining Useful Life with Transformer Ensemble Model, *IEEE Trans. Transp. Electrification* (2024), 1–1.
- [9] J. Liang, H. Liu, N.-C. Xiao, A hybrid approach based on deep neural network and double exponential model for remaining useful life prediction, *Expert Syst. Appl.* 249 (2024).
- [10] L. Wang, H. Cao, Z. Ye, H. Xu, J. Yan, DVGFormer: A dual-view graph Transformer to fuse multi-sensor signals for remaining useful life prediction, *Mech. Syst. Signal Process.* 207 (2024).
- [11] R. Ma, T. Han, W. Lei, Cross-domain meta learning fault diagnosis based on multi-scale dilated convolution and adaptive relation module, *Knowl. Based Syst.* 261 (2023).
- [12] Y. Wang, L. Chen, Y. Liu, L. Gao, Wavelet-Prototypical Network Based on Fusion of Time and Frequency Domain for Fault Diagnosis, *Sens. (Basel)* 21 (2021).
- [13] Y. Feng, J. Chen, J. Xie, T. Zhang, H. Lv, T. Pan, Meta-learning as a promising approach for few-shot cross-domain fault diagnosis: Algorithms, applications, and prospects, *Knowl. Based Syst.* 235 (2022).
- [14] B. Yang, Y. Lei, F. Jia, S. Xing, An intelligent fault diagnosis approach based on transfer learning from laboratory bearings to locomotive bearings, *Mech. Syst. Signal Process.* 122 (2019) 692–706.
- [15] J. Chen, R. Huang, Z. Chen, W. Mao, W. Li, Transfer learning algorithms for bearing remaining useful life prediction: A comprehensive review from an industrial application perspective, *Mech. Syst. Signal Process.* 193 (2023).
- [16] P.R. d O. da Costa, A. Akçay, Y. Zhang, U. Kaymak, Remaining useful lifetime prediction via deep domain adaptation, *Reliab. Eng. Syst. Saf.* 195 (2020).
- [17] Y. Zou, S. Zhao, Y. Liu, Z. Li, X. Song, G. Ding, The Transfer Prediction Method of Bearing Remain Use Life Based on Dynamic Benchmark, *IEEE Trans. Instrum. Meas.* 70 (2021) 1–11.
- [18] H. Su, L. Xiang, A. Hu, Y. Xu, X. Yang, A novel method based on meta-learning for bearing fault diagnosis with small sample learning under different working conditions, *Mech. Syst. Signal Process.* 169 (2022).
- [19] X. Huang, D. Wu, B. Boulet, MetaProformer for Charging Load Probabilistic Forecasting of Electric Vehicle Charging Stations, *IEEE Trans. Intell. Transp. Syst.* (2023) 1–11.
- [20] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks. International conference on machine learning, PMLR, 2017, pp. 1126–1135.
- [21] A. Nichol, J. Achiam, and J. Schulman, On first-order meta-learning algorithms, *arXiv preprint arXiv:02999*. (2018).
- [22] A. Gulati *et al.*, Conformer: Convolution-augmented transformer for speech recognition, (2020).
- [23] Z. Xu, Y. Zhang, Q. Miao, An attention-based multi-scale temporal convolutional network for remaining useful life prediction, *Reliab. Eng. Syst. Saf.* 250 (2024).
- [24] J. Guo, S. Lei, B. Du, MHT: A multiscale hourglass-transformer for remaining useful life prediction of aircraft engine, *Eng. Appl. Artif. Intell.* 128 (2024).
- [25] L. Zhou, H. Wang, S. Xu, Joint learning strategy of multi-scale multi-task convolutional neural network for aero-engine prognosis, *Appl. Soft Comput.* 160 (2024).
- [26] J. Li, *et al.*, A Dual-Scale Transformer-Based Remaining Useful Life Prediction Model in Industrial Internet of Things, *IEEE Internet Things J.* 11 (2024) 26656–26667.
- [27] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K.J. a p a Santosh, Advances and Challenges in Meta-Learning: A Technical Review, (2023).
- [28] h Lee, Meta Learning for Natural Language Processing: A Survey, (2022).
- [29] J. Yang, X. Wang, Z. Luo, Few-shot remaining useful life prediction based on meta-learning with deep sparse kernel network, *Inf. Sci.* 653 (2024).
- [30] Y. Mo, L. Li, B. Huang, X. Li, Few-shot RUL estimation based on model-agnostic meta-learning, *J. Intell. Manuf.* (2022).
- [31] T. Pan, J. Chen, Z. Ye, A. Li, A multi-head attention network with adaptive meta-transfer learning for RUL prediction of rocket engines, *Reliab. Eng. Syst. Saf.* 225 (2022).
- [32] J. Yang, X.J.R.E. Wang, S. Safety, Meta-learning with deep flow kernel network for few shot cross-domain remaining useful life prediction, *Reliab. Eng. Syst. Saf.* 244 (2024) 109928.
- [33] B. Zhang, X. Liu, C. Yue, S.Y. Liang, L. Wang, Meta-learning-based approach for tool condition monitoring in multi-condition small sample scenarios, *Mech. Syst. Signal Process.* 216 (2024).
- [34] L. Chang, Y.-H. Lin, Few-shot remaining useful life prediction based on Bayesian meta-learning with predictive uncertainty calibration, *Eng. Appl. Artif. Intell.* 142 (2025).
- [35] P. Guo, *et al.*, Recent developments on espnet toolkit boosted by conformer. ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2021, pp. 5874–5878.
- [36] A. Vaswani *et al.*, Attention is all you need, *Advances in neural information processing systems*. 30 (2017).
- [37] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: synthetic minority over-sampling technique 16 (2002) 321–357.
- [38] X.Y. Liu, Z.H. Zhou, Ensemble methods for class imbalance learning, in: *Imbalanced learning: Foundations, algorithms, and applications*, 2013, pp. 61–82.
- [39] P. Ding, M. Jia, Y. Ding, X. Zhao, Statistical Alignment-Based Metagated Recurrent Unit for Cross-Domain Machinery Degradation Trend Prognostics Using Limited Data, *IEEE Trans. Instrum. Meas.* 70 (2021) 1–12.
- [40] A. Saxena, K. Goebel, D. Simon, N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation. 2008 international conference on prognostics and health management, IEEE, 2008, pp. 1–9.
- [41] N. Costa, L. Sánchez, Variational encoding approach for interpretable assessment of remaining useful life estimation, *Reliab. Eng. Syst. Saf.* 222 (2022).
- [42] X. Liao, S. Chen, P. Wen, S. Zhao, Remaining useful life with self-attention assisted physics-informed neural network, *Adv. Eng. Inform.* 58 (2023).
- [43] F.O. Heimes, "Recurrent neural networks for remaining useful life estimation. 2008 international conference on prognostics and health management, IEEE, 2008, pp. 1–6.
- [44] H. Li, W. Zhao, Y. Zhang, E. Zio, Remaining useful life prediction using multi-scale deep convolutional neural network, *Appl. Soft Comput.* 89 (2020).
- [45] J. Xia, Y. Feng, C. Lu, C. Fei, X. Xue, LSTM-based multi-layer self-attention method for remaining useful life estimation of mechanical systems, *Eng. Fail. Anal.* 125 (2021) 105385.
- [46] I. de Pater, A. Reijns, M.J.R.E. Mitici, and S. Safety, Alarm-based predictive maintenance scheduling for aircraft engines with imperfect Remaining Useful Life prognostics, 221 (2022) 108341.
- [47] A.L. Ellefsen, E. Bjørlykhaug, V. Æsøy, S. Ushakov, H.J.R.E. Zhang, and S. Safety, Remaining useful life predictions for turbofan engine degradation using semi-supervised deep architecture, 183 (2019) 240–251.
- [48] Y. Zhang, C. Zhang, S. Wang, H. Dui, R. Chen, Health indicators for remaining useful life prediction of complex systems based on long short-term memory network and improved particle filter, *Reliab. Eng. Syst. Saf.* 241 (2024).
- [49] T. Zhang, L. Jiang, R. Huang, X. Zhang, A Multiscale Cross-Channel Attention Network for Remaining Useful Life Prediction With Variable Sensors, *IEEE Trans. Instrum. Meas.* 74 (2025) 1–10.
- [50] C. Cai, Y. Fang, W. Liu, R. Jin, J. Cheng, Z. Chen, FedCov: Enhanced Trustworthy Federated Learning for Machine RUL Prediction With Continuous-to-Discrete Conversion, *IEEE Trans. Ind. Inform.* 20 (2024) 12565–12574.
- [51] S. Fu, Y. Zhang, L. Lin, M. Zhao, S.-s J.R.E. Zhong, and S. Safety, Deep residual LSTM with domain-invariance for remaining useful life prediction across domains, 216 (2021) 108012.
- [52] M. Ragab, *et al.*, Contrastive Adversarial Domain Adaptation for Machine Remaining Useful Life Prediction, *IEEE Trans. Ind. Inform.* 17 (2021) 5239–5249.
- [53] J. Zhang, X. Li, J. Tian, Y. Jiang, H. Luo, S. Yin, A variational local weighted deep sub-domain adaptation network for remaining useful life prediction facing cross-domain condition, *Reliab. Eng. Syst. Saf.* 231 (2023).
- [54] X. Li, Q. Ding, J.-Q. Sun, Remaining useful life estimation in prognostics using deep convolution neural networks, *Reliab. Eng. Syst. Saf.* 172 (2018) 1–11.